ESPRIT · Module MT-41 · Applications Web Distribuées

Mini-projet 2026-2027 · Validation intermédiaire (séance 4)

Refugia

Dossier technique

Application de gestion d'un refuge pour animaux en architecture microservices

Classe 4DAS_I13 · Groupe InSync 3

Microservices : Animaux · Demandes d'adoption · Familles d'accueil · Suivi vétérinaire · Dons et parrainages · Visites et rendez-vous · Notifications

Novembre 2026

Sommaire

1. Présentation du projet 3
1.1 Contexte et problématique 3
1.2 Objectifs 3
1.3 Périmètre 3

2. Acteurs et fonctionnalités 4
2.1 Acteurs 4
2.2 Diagramme général des acteurs et des cas d'utilisation 4
2.3 Fonctionnalités principales 5
2.4 Scénarios métier principaux 5

3. Architecture globale 6
3.1 Diagramme d'architecture 6
3.2 Composants 6
3.3 Routage de l'API Gateway 7
3.4 Sécurité 7
3.5 Données 8
3.6 Communications entre services 8

4. Découpage en microservices 9

5. Conventions communes 10
5.1 API REST 10
5.2 Messages RabbitMQ 10
5.3 Structure d'un microservice 10

6. Conception détaillée de chaque microservice 12
6.1 MS1 · Animaux 12
6.2 MS2 · Demandes d'adoption 17
6.3 MS3 · Familles d'accueil 22
6.4 MS4 · Suivi vétérinaire 26
6.5 MS5 · Dons et parrainages 30
6.6 MS6 · Visites et rendez-vous 34
6.7 MS7 · Notifications (service commun) 39

7. Interactions entre les services 43
7.1 Appels synchrones 43
7.2 Catalogue des événements 43
7.3 Scénario de bout en bout : adopter un animal 44

8. Technologies choisies 45

9. Répartition des tâches 46

10. Organisation du dépôt Git et gestion de projet 46
10.1 Dépôt 46
10.2 Méthode de travail 47
10.3 Outil de gestion de projet 47

11. Planning prévisionnel 47

12. Principaux risques 48

13. Conclusion 48

Annexe A · Récapitulatif des services 49

Annexe B · Glossaire 49

1. Présentation du projet

1.1 Contexte et problématique

Un refuge pour animaux gère en même temps de nombreuses activités : l'accueil et le suivi des animaux, les demandes d'adoption, les familles d'accueil, les soins vétérinaires, les dons, les visites du public et les événements. Dans beaucoup de refuges, ces informations sont dispersées entre des fiches papier, des tableurs et des messages. Il en résulte des animaux dont le statut n'est pas à jour, des demandes qui se perdent, des soins oubliés et des donateurs qui ne reçoivent pas de reçu.

Problématique : comment centraliser la gestion d'un refuge dans une application qui reste simple à faire évoluer, où chaque domaine métier (animaux, adoptions, accueil, soins, dons, visites) est géré de façon autonome, tout en échangeant avec les autres sans dépendre d'eux ?

Refugia répond à cette question par une application web construite en **architecture microservices** : chaque domaine est un service Spring Boot indépendant, avec sa propre base de données, et les services communiquent par des appels REST synchrones et par des événements asynchrones.

1.2 Objectifs

- Offrir au public un catalogue des animaux à adopter et la possibilité de réserver une visite, de déposer une demande d'adoption, de faire un don ou de parrainer un animal.
- Donner au personnel du refuge un outil unique pour suivre les animaux, traiter les demandes, organiser les visites et les événements, et suivre la santé des animaux.
- Garantir que chaque service puisse être développé, testé et déployé séparément (une base par service, aucune lecture directe dans la base d'un autre).
- Respecter la stack imposée par le module MT-41 : Spring Boot, Spring Cloud Gateway, Eureka, Keycloak, Docker, une base de données par service et un microservice commun avec une technologie avancée (MongoDB).

1.3 Périmètre

Le projet couvre sept microservices (six services individuels et un service commun), une API Gateway, un registre Eureka, un serveur Keycloak, un broker RabbitMQ et une application React.

Inclus dans le projet	Hors périmètre
Gestion des fiches animaux, de leur statut et de leur historique	Paiement en ligne réel (le don enregistre un mode de paiement, sans passerelle bancaire)
Demandes d'adoption avec score de compatibilité	Gestion comptable du refuge
Familles d'accueil et placements	Gestion des stocks (nourriture, médicaments)
Dossiers médicaux, vaccinations, traitements, rappels	Gestion des ressources humaines et des bénévoles
Dons, reçus, parrainages mensuels, statistiques de dons	Application mobile native
Visites sur créneaux, événements, liste d'attente	
Notifications par e-mail et internes	

2. Acteurs et fonctionnalités

2.1 Acteurs

L'application distingue trois rôles, gérés dans Keycloak. Le vétérinaire est un membre du personnel qui appartient en plus au groupe Keycloak « veterinaire » : ce groupe lui donne le droit d'écrire dans le suivi vétérinaire (MS4), que les autres membres du personnel ne peuvent que consulter.

Rôle	Qui	Ce qu'il fait
`CLIENT`	Visiteur, adoptant, donateur, parrain, famille d'accueil	Consulte les animaux, réserve une visite, s'inscrit à un événement, dépose et suit une demande d'adoption, fait un don, parraine un animal, consulte ses notifications
`PERSONNEL`	Employés du refuge et vétérinaire	Gère les fiches animaux et leur statut, traite les demandes d'adoption, gère les familles d'accueil et les placements, organise les créneaux et les événements, consulte les dons et les statistiques ; le vétérinaire gère en plus les dossiers médicaux
`ADMIN`	Responsable du refuge	Possède tous les droits du personnel ; en plus : supprime une fiche saisie par erreur, consulte les statistiques des animaux, annule un événement, annule un don, renvoie une notification et gère les comptes et les rôles dans Keycloak

Un visiteur sans compte peut consulter le catalogue des animaux, les créneaux de visite disponibles et les événements à venir. Toute autre action demande une connexion.

2.2 Diagramme général des acteurs et des cas d'utilisation

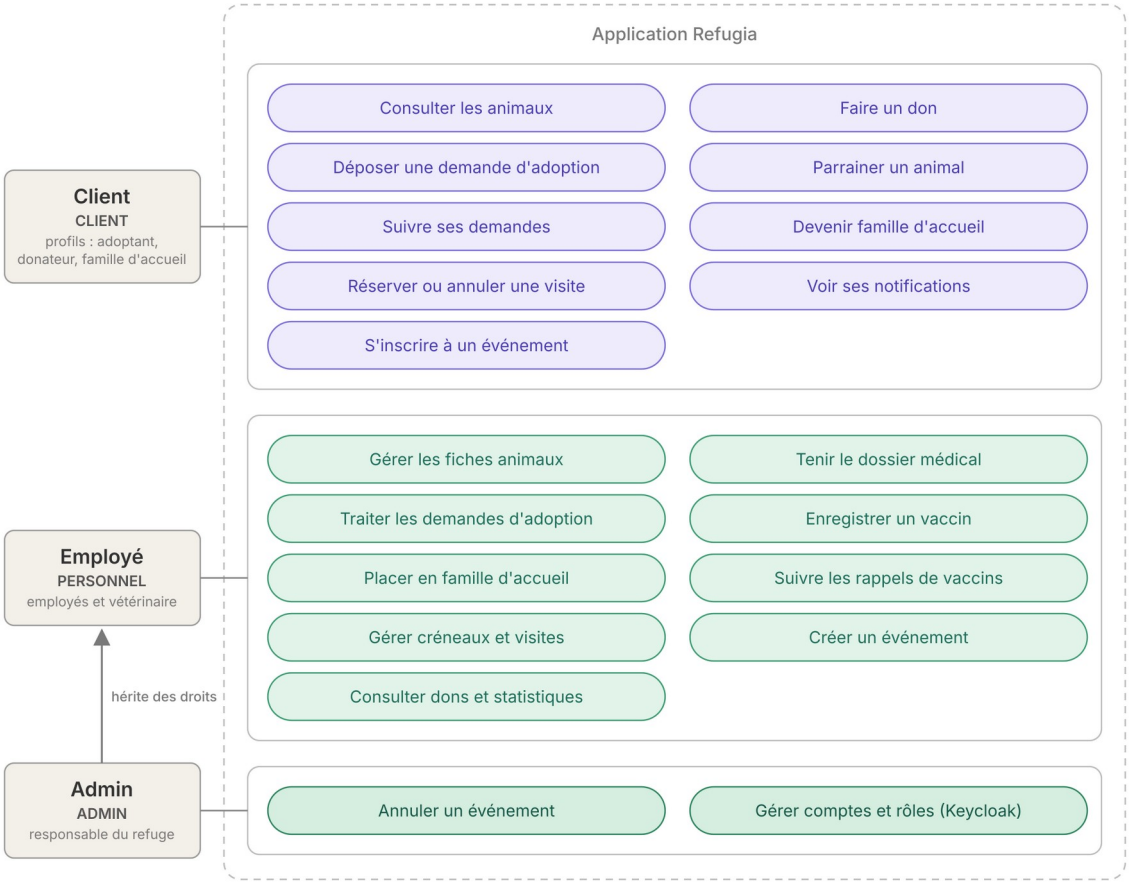


Figure 1 : Acteurs et actions de l'application Refugia (vue générale)

2.3 Fonctionnalités principales

Domaine	Fonctionnalités	Service
Animaux	Création et modification des fiches, recherche et filtres, statut avec transitions contrôlées, photos, historique des statuts, statistiques	MS1
Adoption	Profil du candidat, demande d'adoption, score de compatibilité, acceptation ou refus, finalisation de l'adoption	MS2
Familles d'accueil	Enregistrement des familles et de leur capacité, placement d'un animal, fin ou annulation d'un placement, historique	MS3
Suivi vétérinaire	Dossier médical par animal, consultations, vaccinations, rappels de vaccin, traitements	MS4
Dons et parrainages	Dons ponctuels avec reçu, parrainages mensuels (suspension, reprise, fin), donateurs, statistiques de dons	MS5
Visites et événements	Créneaux de visite, réservation et annulation, événements (journée d'adoption, atelier), inscriptions et liste d'attente	MS6
Notifications	Notifications par e-mail et internes déclenchées par les événements des autres services, consultation et renvoi	MS7

2.4 Scénarios métier principaux

- **Adopter un animal.** Un client consulte le catalogue (MS1), réserve une visite (MS6), dépose une demande d'adoption avec son profil (MS2). Le personnel consulte le score de compatibilité, accepte la demande puis la finalise : l'animal passe au statut ADOPTE (MS1), ses parrainages se terminent (MS5), son éventuel placement se termine (MS3) et le client est prévenu (MS7).
- **Faire un don ou parrainer.** Un client fait un don pour le refuge ou pour un animal précis ; le reçu est généré et envoyé par e-mail (MS5, MS7). Il peut aussi parrainer un animal chaque mois.
- **Placer un animal en famille d'accueil.** Le personnel choisit une famille active qui a une place libre et accepte l'espèce ; l'animal passe en accueil (MS3, MS1).
- **Soigner un animal.** Le vétérinaire ouvre le dossier médical, enregistre consultations, vaccins et traitements ; un rappel de vaccin est notifié au personnel quelques jours avant l'échéance (MS4, MS7).
- **Organiser une journée d'adoption.** Le personnel crée un événement avec un nombre de places ; au-delà, les inscrits passent en liste d'attente et sont promus automatiquement si une place se libère (MS6).

3. Architecture globale

3.1 Diagramme d'architecture

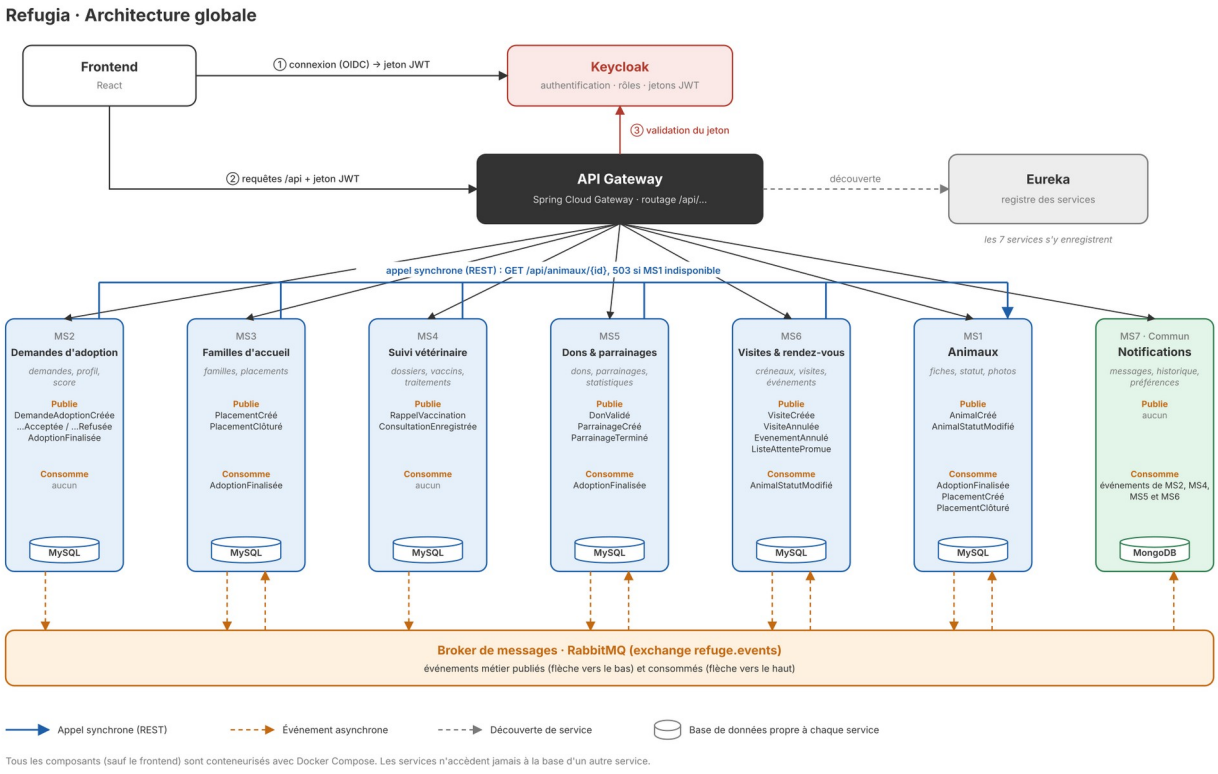


Figure 2 : Architecture globale de Refugia

3.2 Composants

Composant	Rôle	Technologie
Frontend	Application web unique pour les trois rôles ; appelle uniquement l'API Gateway ; connecte l'utilisateur à Keycloak	React
API Gateway	Point d'entrée unique : routage vers les services, validation du jeton JWT, routes publiques en lecture seule	Spring Cloud Gateway
Registre de services	Les services s'enregistrent au démarrage ; la Gateway et les appels entre services trouvent les adresses par le nom du service	Eureka
Sécurité	Authentification OIDC, émission des jetons JWT, gestion des comptes, des rôles et du groupe « veterinaire »	Keycloak
Microservices métier	Sept services Spring Boot, chacun avec sa base ; documentation Swagger/OpenAPI ; enregistrés dans Eureka	Spring Boot, Spring Data JPA
Broker de messages	Transporte les événements entre services ; exchange `refuge.events` de type topic ; une file par service consommateur	RabbitMQ
Bases de données	Une base par service : MySQL pour MS1 à MS6, MongoDB pour MS7	MySQL, MongoDB
Conteneurisation	Chaque composant dans un conteneur ; un fichier unique démarre l'ensemble	Docker, Docker Compose

3.3 Routage de l'API Gateway

Chemin exposé	Service	Accès sans compte
`/api/animaux/**`	MS1 Animaux	GET catalogue et détail
`/api/demandes/**`, `/api/profils/**`	MS2 Demandes d'adoption	Non
`/api/familles/**`, `/api/placements/**`	MS3 Familles d'accueil	Non
`/api/vet/**`	MS4 Suivi vétérinaire	Non
`/api/donateurs/**`, `/api/dons/**`, `/api/parrainages/**`, `/api/stats/**`	MS5 Dons et parrainages	Non
`/api/visites/**`, `/api/evenements/**`	MS6 Visites et rendez-vous	GET créneaux et événements
`/api/notifications/**`	MS7 Notifications	Non

> Le chemin `GET /api/animaux/{animalId}/parrains` appartient à MS5 alors qu'il commence comme les routes de MS1. La Gateway déclare donc cette route plus précise en premier, pour qu'elle soit servie par MS5 et non par MS1.

3.4 Sécurité

- **Authentification.** L'utilisateur se connecte à Keycloak (flux OIDC avec PKCE depuis React) et reçoit un jeton JWT qui contient son identifiant et ses rôles.
- **Contrôle à la Gateway.** La Gateway vérifie la signature et la durée du jeton pour toute route non publique et rejette avec 401 un jeton absent ou invalide.
- **Contrôle dans chaque service.** Chaque service vérifie aussi le jeton et applique ses règles par rôle (403 si le rôle est insuffisant). Une requête qui contournerait la Gateway reste donc refusée.
- **Appels entre services.** Quand un service en appelle un autre (par exemple MS6 vers MS1), il transmet le jeton de l'utilisateur à l'origine de l'appel ; MS1 applique alors les droits de cet utilisateur.
- **Messages RabbitMQ.** Les services se connectent au broker avec leur propre compte. Un message reçu n'a pas d'utilisateur : il est traité au nom du service.
- **Rôle ADMIN.** Dans Keycloak, ADMIN est un rôle composite qui contient PERSONNEL : il hérite donc de tous ses droits.

Droits par rôle et par service

Service	Sans compte	CLIENT	PERSONNEL	ADMIN en plus
MS1 Animaux	Consulte le catalogue	Consulte	Crée, modifie, change le statut, gère les photos, consulte l'historique	Supprime une fiche, statistiques
MS2 Demandes	–	Gère son profil et ses demandes	Traite, accepte, refuse, finalise, consulte les scores	Supervise
MS3 Familles	–	–	Gère les familles et les placements	Mêmes droits
MS4 Vétérinaire	–	–	Consulte ; le vétérinaire (groupe « veterinaire ») écrit	Consulte et écrit
MS5 Dons	–	Fait des dons et parrainages, consulte les siens	Consulte dons, parrainages et statistiques	Annule un don, modifie un donateur
MS6 Visites	Consulte créneaux et événements	Réserve, annule ses visites, s'inscrit aux événements	Gère les créneaux et les événements, consulte les inscrits	Annule un événement
MS7 Notifications	–	Consulte ses notifications	Consulte ses notifications et celles du personnel	Liste complète, renvoi

3.5 Données

- **Une base par service.** MS1 à MS6 utilisent MySQL, chacun dans un schéma séparé ; MS7 utilise MongoDB, un choix de « technologie avancée » adapté à des messages de formes différentes.
- **Aucune clé étrangère entre services.** Un service qui parle d'un animal garde uniquement son identifiant `animalId` ; pour le reste, il interroge MS1.
- **Copies limitées.** Quand un service doit afficher des informations stables d'un autre (par exemple MS6 qui garde `nomAnimal` et `espece` au moment d'une réservation), il les copie à ce moment-là, sans les mettre à jour ensuite.
- **Environnement de test.** Les tests d'intégration utilisent H2 en mémoire, comme dans l'exemple vu en cours.

3.6 Communications entre services

	Communication synchrone	Communication asynchrone
Technologie	REST avec OpenFeign, via Eureka	RabbitMQ (exchange topic `refuge.events`)
Quand l'utiliser	L'appelant a besoin de la réponse tout de suite pour décider (existence ou statut d'un animal)	L'émetteur informe d'un fait sans attendre de réponse (don validé, adoption finalisée)
Si l'autre service est arrêté	Réponse 503 et rien n'est enregistré (délai maximal 2 s, coupe-circuit Resilience4j)	Le message attend dans la file et est traité au redémarrage
Exemples	MS6, MS5, MS4, MS3, MS2 vers MS1 : GET `/api/animaux/{id}`	MS2 publie `AdoptionFinalisée` ; MS5, MS3 et MS7 la reçoivent

Pour qu'un message livré deux fois ne produise pas deux effets, chaque consommateur ignore un événement dont l'identifiant `eventId` a déjà été traité (idempotence).

4. Découpage en microservices

Le découpage suit les domaines métier : un service est responsable d'un seul domaine et de ses données. MS1 est le service de référence sur les animaux ; les autres services ne connaissent un animal que par son identifiant.

Service	Responsabilité	Base	Appelle (sync)	Publie	Écoute
MS1 Animaux	Fiches, statuts, photos, historique	MySQL	–	AnimalCréé, StatutAnimalModifié	–
MS2 Demandes d'adoption	Profil candidat, demandes, score, finalisation	MySQL	MS1	4 événements d'adoption	–
MS3 Familles d'accueil	Familles, capacité, placements	MySQL	MS1	PlacementCréé, PlacementTerminé	AdoptionFinalisée
MS4 Suivi vétérinaire	Dossier médical, consultations, vaccins, traitements	MySQL	MS1	RappelVaccination, ConsultationEnregistrée	–
MS5 Dons et parrainages	Donateurs, dons, reçus, parrainages, statistiques	MySQL	MS1	DonValidé, ParrainageCréé, ParrainageTerminé	AdoptionFinalisée
MS6 Visites et rendez-vous	Créneaux, visites, événements, inscriptions	MySQL	MS1	4 événements de visites	StatutAnimalModifié
MS7 Notifications (commun)	Notifications e-mail et internes	MongoDB	–	–	Tous les événements

Pourquoi ce découpage. Chaque service peut être compris, testé et déployé seul. La dépendance est à sens unique : tous les services dépendent de MS1 pour connaître un animal, et MS1 ne dépend d'aucun autre. Le service commun MS7 n'appelle personne : il n'a besoin que des événements.

Service commun. Conformément au sujet, MS7 est développé par toute l'équipe : chaque membre ajoute dans MS7 l'écouteur et le gabarit de message des événements de son propre service. Un seul membre le présente lors de la soutenance.

5. Conventions communes

Pour que les sept services soient cohérents, l'équipe applique les mêmes règles.

5.1 API REST

- Préfixe `/api` pour toutes les routes, format JSON, dates au format ISO 8601.
- Documentation Swagger/OpenAPI sur chaque service (`/swagger-ui.html`).
- Un GET de liste accepte `page` et `size` quand la liste peut être longue.
- Les identifiants d'utilisateur (`clientId`, `visiteurId`, etc.) sont ceux du jeton Keycloak : le service les lit dans le jeton, jamais dans le corps de la requête.

Codes de réponse

Code	Signification
200 / 201 / 204	Succès, création, succès sans contenu
400	Données invalides (champ manquant, valeur hors règle, date incorrecte)
401 / 403	Non authentifié (jeton absent ou invalide) / rôle insuffisant ou ressource d'un autre utilisateur
404	Ressource introuvable (y compris un animal inconnu de MS1)
409	Conflit avec une règle métier (doublon, capacité atteinte, transition interdite)
503	Un service appelé ne répond pas (MS1 arrêté ou plus lent que 2 s)

Format d'erreur commun

Toutes les erreurs 4xx et 5xx ont le même corps JSON, géré par une classe `GlobalExceptionHandler` dans chaque service :

```
{
  "timestamp": "2026-11-11T10:30:00",
  "status": 404,
  "error": "Not Found",
  "message": "Animal 5 introuvable",
  "path": "/api/dons"
}
```

5.2 Messages RabbitMQ

- Un seul exchange de type topic : `refuge.events`.
- **Clé de routage** en minuscules, mots séparés par des points : `adoption.finalisee`, `visite.creee`.
- **Enveloppe commune** : `eventId` (identifiant unique), `eventType` (nom de l'événement), `occurredAt` (date et heure) et `data` (contenu).
- **File par consommateur** : chaque service qui écoute un événement a sa propre file, nommée `<service>.<clé>` (exemple : `ms6.animal.statut.modifie`).
- **Idempotence** : un consommateur ignore un `eventId` déjà traité.

5.3 Structure d'un microservice

Tous les services suivent la structure de l'exemple vu en cours (projet `condidat_s1`) : Spring Boot 3.5, Java 17, Maven, Spring Data JPA, validation, MySQL et springdoc-openapi.

```
msX-nom/
├── database/                        données de test (script SQL)
├── src/main/java/com/awd/<service>/
│   ├── controller/                  endpoints REST
│   ├── service/                     logique métier
│   ├── repository/                  accès à la base
│   ├── entity/                      entités (tables)
│   ├── dto/                         objets Request et Response
│   ├── mapper/                      conversion entité <-> DTO
│   └── common/{config, exception}/  Swagger, gestion des erreurs
├── src/main/resources/application.properties
├── src/test/java/                   tests d'intégration
├── pom.xml
└── README.md
```

Le `application.properties` de chaque service définit son nom (`spring.application.name`), son port, sa base de données et le chemin de Swagger. Les services d'événements ajoutent les packages `client` (appels OpenFeign) et `event` (publication et réception RabbitMQ).

6. Conception détaillée de chaque microservice

Chaque fiche présente le rôle du service, ses données, ses règles, son diagramme de classes, ses échanges avec les autres services (un exemple synchrone et un exemple asynchrone) et son contrat d'API : endpoints, rôles, codes de réponse et événements.

6.1 MS1 · Animaux

Rôle

MS1 gère la fiche de chaque animal du refuge et son statut. C'est le service de référence du projet : MS2 à MS6 ne connaissent un animal que par son identifiant (`animalId`) et lisent MS1 pour le reste. MS1 n'appelle aucun autre service en synchrone.

- **Fiches** : création, modification, consultation et recherche des animaux.
- **Statut** : situation actuelle de l'animal (disponible, réservé, en accueil, en soins, adopté), avec contrôle des transitions.
- **Photos** : images rattachées à une fiche.
- **Historique** : trace de chaque changement de statut et de son origine.

Données (base MySQL {{animal_db}})

Entité	Attributs principaux
Animal	id, nom, espece, race, sexe, dateNaissance, taille, description, niveauEnergie, ententeEnfants, ententeAutresAnimaux, statut, dateArrivee
Photo	id, animalId, url, principale, dateAjout
HistoriqueStatut	id, animalId, ancienStatut, nouveauStatut, dateChangement, origine, motif

Énumération	Valeurs
StatutAnimal	DISPONIBLE, RESERVE, EN_ACCUEIL, EN_SOINS, ADOPTE
Espece	CHIEN, CHAT, LAPIN, OISEAU, AUTRE
Sexe / Taille	MALE, FEMELLE / PETIT, MOYEN, GRAND

`niveauEnergie` (de 1 à 5), `ententeEnfants` et `ententeAutresAnimaux` servent au score de compatibilité calculé par MS2. `origine` indique qui a déclenché un changement : l'identifiant d'un membre du personnel ou le nom du service appelant.

Règles métier

- Une fiche créée reçoit le statut DISPONIBLE, sauf si le personnel indique EN_SOINS à l'arrivée (quarantaine). Nom, espèce, sexe et date d'arrivée sont obligatoires ; la date de naissance ne peut pas être dans le futur (400).
- Seul l'ADMIN supprime une fiche, pour corriger une erreur de saisie. Un animal ADOPTE ou EN_ACCUEIL ne peut pas être supprimé (409).
- Le catalogue public montre les animaux DISPONIBLE, RESERVE et EN_ACCUEIL ; le personnel voit tous les statuts.
- Le statut ne change que par les transitions du tableau ci-dessous ; toute autre demande renvoie 409, y compris demander le statut déjà en cours.
- Le passage à ADOPTE est demandé par MS2 à la finalisation d'une adoption (appel synchrone, voir 7.1), jamais saisi à la main. Le passage à EN_ACCUEIL et le retour à DISPONIBLE sont demandés par MS3.

- Chaque changement ajoute une ligne dans HistoriqueStatut et publie StatutAnimalModifié, dans la même transaction. L'historique est en lecture seule.
- Cinq photos au plus par animal (409 au-delà), dont une seule principale.

Statut actuel	Statuts suivants autorisés
DISPONIBLE	RESERVE, EN_ACCUEIL, EN_SOINS, ADOPTE
RESERVE	DISPONIBLE, ADOPTE
EN_ACCUEIL	DISPONIBLE, RESERVE, EN_SOINS, ADOPTE
EN_SOINS	DISPONIBLE, EN_ACCUEIL
ADOPTE	Aucun (état final)

Diagramme de classes

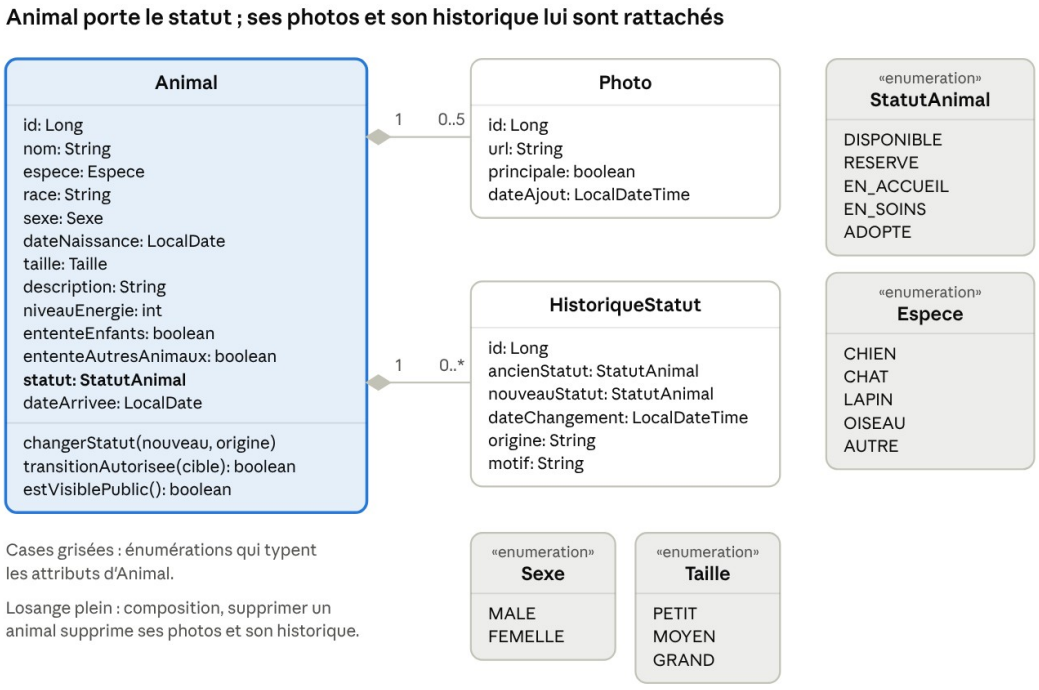


Figure 3 : MS1 · modèle de domaine (3 classes, 4 énumérations)

Couches applicatives : `AnimalController` et `PhotoController` (endpoints), `AnimalService` et `PhotoService` (règles métier, contrôle des transitions, historique), `AnimalRepository`, `PhotoRepository` et `HistoriqueStatutRepository` (accès JPA) et `AnimalEventPublisher` (publication de `AnimalCréé` et `StatutAnimalModifié`). DTO : AnimalRequest, AnimalResponse, StatutRequest, PhotoRequest, HistoriqueResponse.

Cas d'utilisation

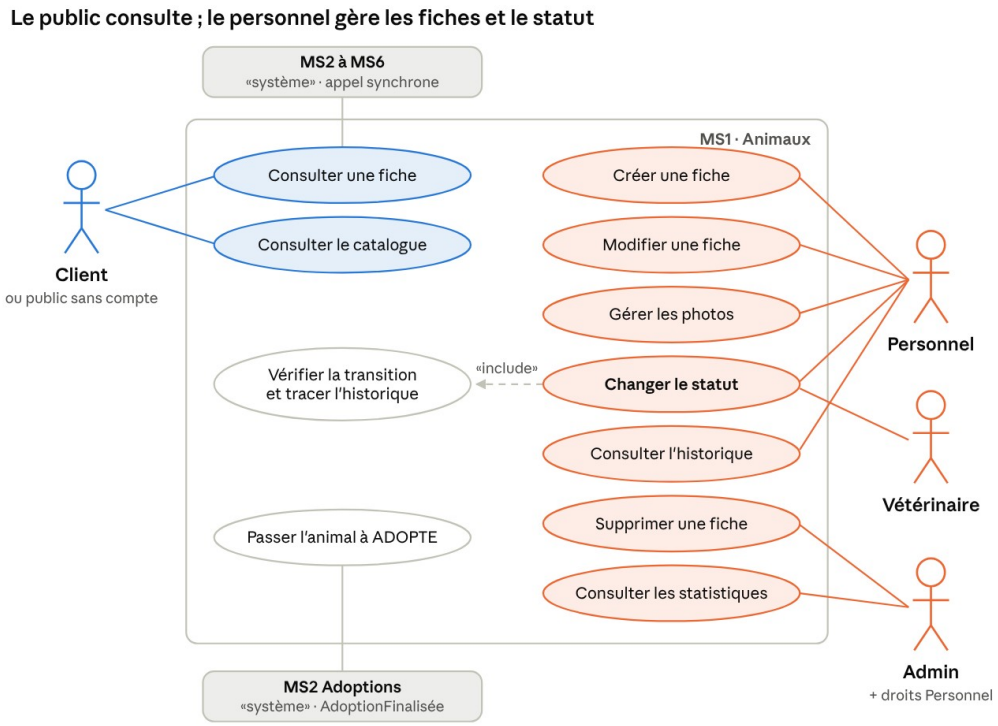


Figure 4 : MS1 · cas d'utilisation

Communication synchrone : MS6 lit la fiche avant de confirmer une visite

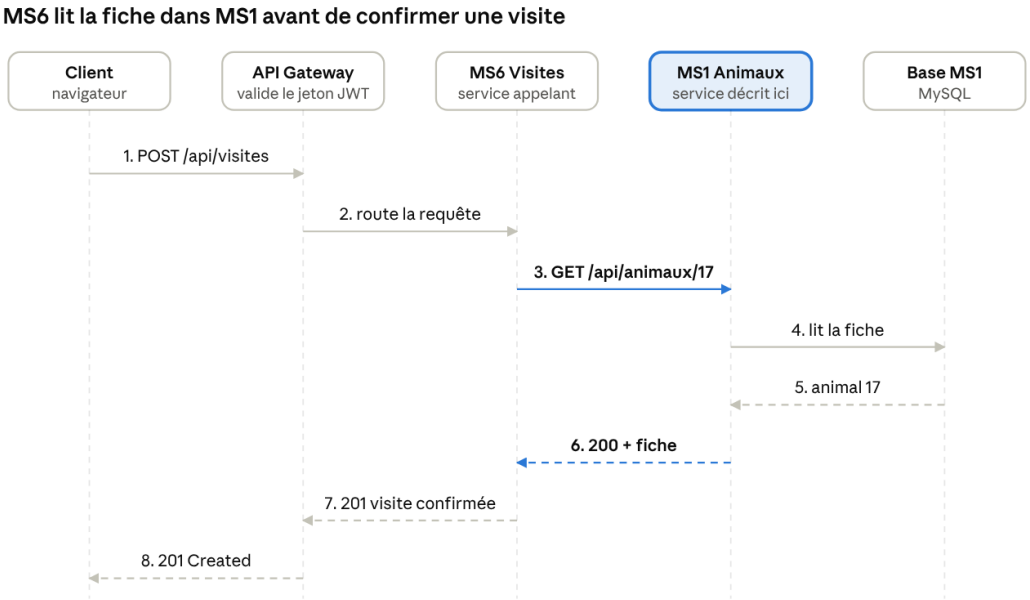


Figure 5 : MS1 · exemple de communication synchrone (appel de MS6)

MS1 renvoie la fiche avec son statut et ne décide pas à la place de l'appelant : c'est le service appelant qui juge si l'animal convient à son cas. Le même appel sert à MS2, MS3, MS4 et MS5.

Situation	Réponse de MS1	Effet chez l'appelant
Animal inconnu	404	L'appelant répond 404

Situation	Réponse de MS1	Effet chez l'appelant
Animal trouvé, statut ADOPTE ou EN_SOINS	200 avec le statut	L'appelant applique sa règle et répond 409 si besoin
MS1 indisponible ou plus lent que 2 s	Aucune	L'appelant répond 503 et n'enregistre rien (coupe-circuit)

Communication asynchrone : un changement de statut

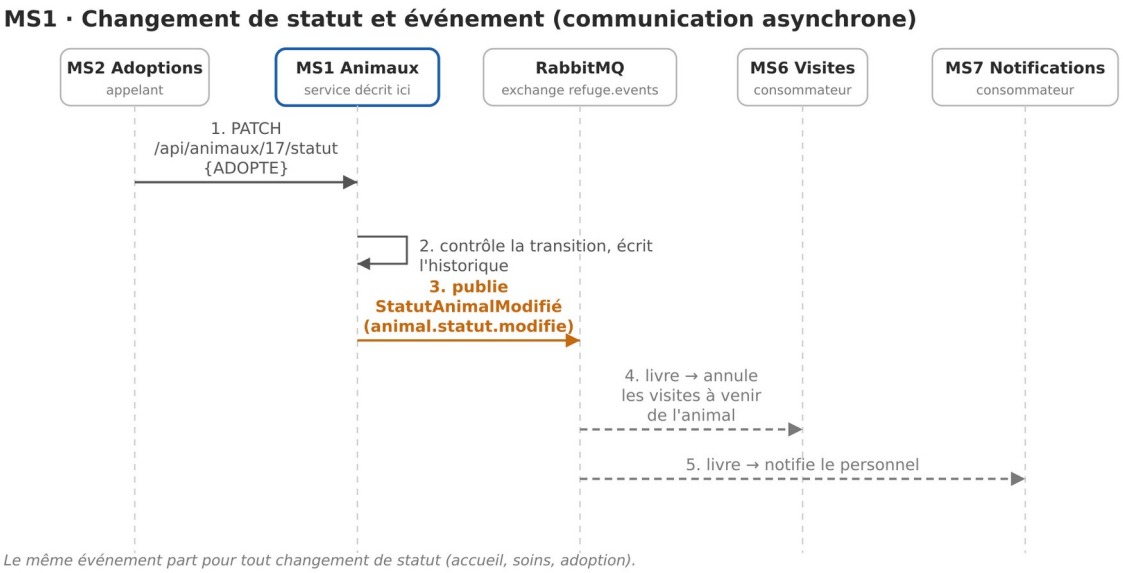


Figure 6 : MS1 · exemple de communication asynchrone (StatutAnimalModifié)

Contrat d'API

Dix endpoints sous le préfixe `/api/animaux`. Codes communs aux endpoints protégés : 401 (non authentifié), 403 (rôle insuffisant).

Méthode	Endpoint	Description	Rôles	Codes
GET	/api/animaux?espece=&statut=&taille=&q=&page=&size=	Lister et filtrer les animaux	Tous	200, 400
GET	/api/animaux/{id}	Détail d'une fiche avec ses photos	Tous, et MS2 à MS6	200, 404
POST	/api/animaux	Créer une fiche (publie AnimalCréé)	PERSONNEL	201, 400
PUT	/api/animaux/{id}	Modifier une fiche, hors statut	PERSONNEL	200, 400, 404
PATCH	/api/animaux/{id}/statut	Changer le statut (publie StatutAnimalModifié) ; appelé aussi par MS2 et MS3	PERSONNEL	200, 400, 404, 409
DELETE	/api/animaux/{id}	Supprimer une fiche saisie par erreur	ADMIN	204, 404, 409
GET	/api/animaux/{id}/historique	Parcours des statuts de l'animal	PERSONNEL	200, 404
POST	/api/animaux/{id}/photos	Ajouter une photo	PERSONNEL	201, 400, 404, 409
DELETE	/api/animaux/{id}/photos/{photoId}	Retirer une photo	PERSONNEL	204, 404
GET	/api/animaux/statistiques	Nombre d'animaux par statut et par espèce, durée moyenne de séjour	ADMIN	200

ADMIN a aussi tous les droits de PERSONNEL. Les deux premières routes sont publiques.

Événement	Clé de routage	Déclencheur	Contenu de data
AnimalCréé	animal.cree	Création d'une fiche	animalId, nom, espece, statut
StatutAnimalModifié	animal.statut.modifie	Changement de statut (accepté)	animalId, ancienStatut, nouveauStatut, origine

6.2 MS2 · Demandes d'adoption

Rôle

MS2 gère les demandes des personnes qui souhaitent adopter un animal. Il possède sa propre base MySQL et ne modifie jamais la base de MS1 : il ne stocke que `animalId`.

- **Demandes d'adoption** : un client choisit un animal, soumet une demande et consulte son état.
- **Profil du candidat** : logement, expérience avec les animaux, disponibilités et préférences.
- **Score de compatibilité** : évalue si le profil du candidat correspond à l'animal choisi, à partir du profil et des caractéristiques de l'animal lues dans MS1 (niveau d'énergie, entente avec les enfants et les autres animaux).
- **Validation** : le personnel accepte ou refuse les demandes ; une demande acceptée peut être finalisée.

Données (base MySQL)

Entité	Attributs principaux
DemandeAdoption	id, animalId, clientId, statut, motifRefus, dateCreation, dateDecision
ProfilCandidat	id, clientId, typeLogement (APPARTEMENT, MAISON), jardin, nbPersonnes, enfants, autresAnimaux, experienceAnimaux, disponibilites
ScoreCompatibilite	id, scoreTotal, dateCalcul

Statuts d'une demande : `EN_ATTENTE`, `EN_ETUDE`, `ACCEPTEE`, `REFUSEE`, `ANNULEE`, `FINALISEE`. `animalId` et `clientId` sont de simples références : MS2 ne stocke aucune donnée de MS1.

Règles métier

- Un client crée et suit ses propres demandes ; le personnel les étudie, les accepte ou les refuse (avec un motif) ; l'ADMIN supervise.
- Une demande ne peut porter que sur un animal qui existe et qui est disponible : sinon 404 ou 409 (« animal déjà adopté ou indisponible »).
- Seule une demande ACCEPTEE peut être finalisée.
- À la finalisation, MS2 demande à MS1 de passer l'animal à ADOPTE ; si MS1 ne répond pas, la finalisation échoue en 503 et la demande reste ACCEPTEE.

Diagramme de classes

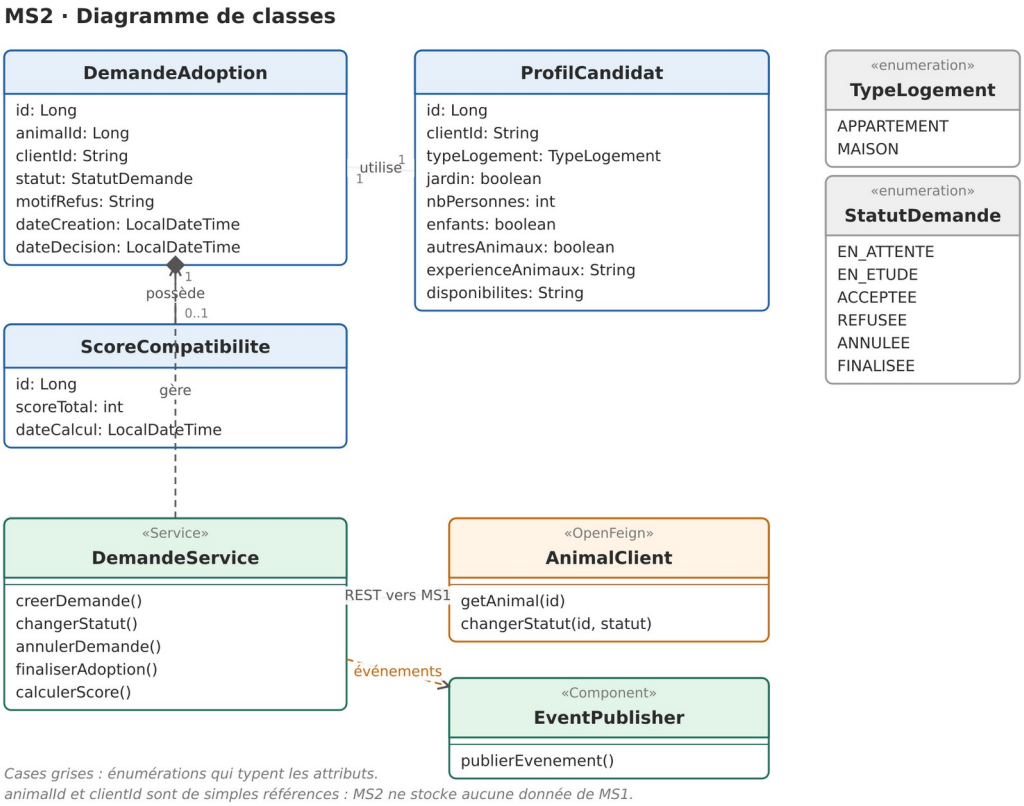


Figure 7 : MS2 · diagramme de classes

`DemandeService` orchestre la logique métier, `AnimalClient` réalise l'appel synchrone vers MS1 et `EventPublisher` publie les événements asynchrones.

Cas d'utilisation

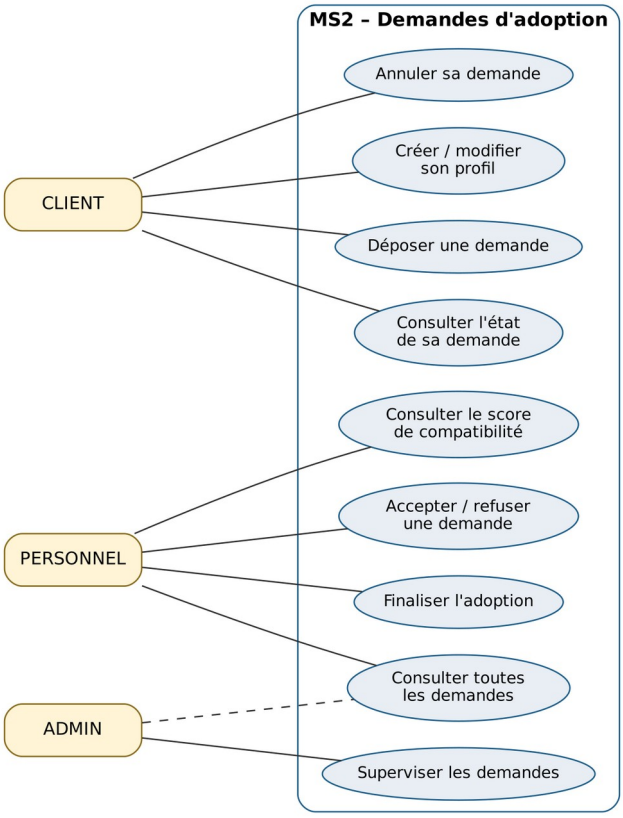


Figure 8 : MS2 · cas d'utilisation

Le CLIENT gère son profil et ses demandes ; le PERSONNEL étudie, accepte, refuse et finalise ; l'ADMIN supervise (le trait pointillé indique qu'il peut aussi consulter toutes les demandes).

Communication synchrone : finaliser une adoption

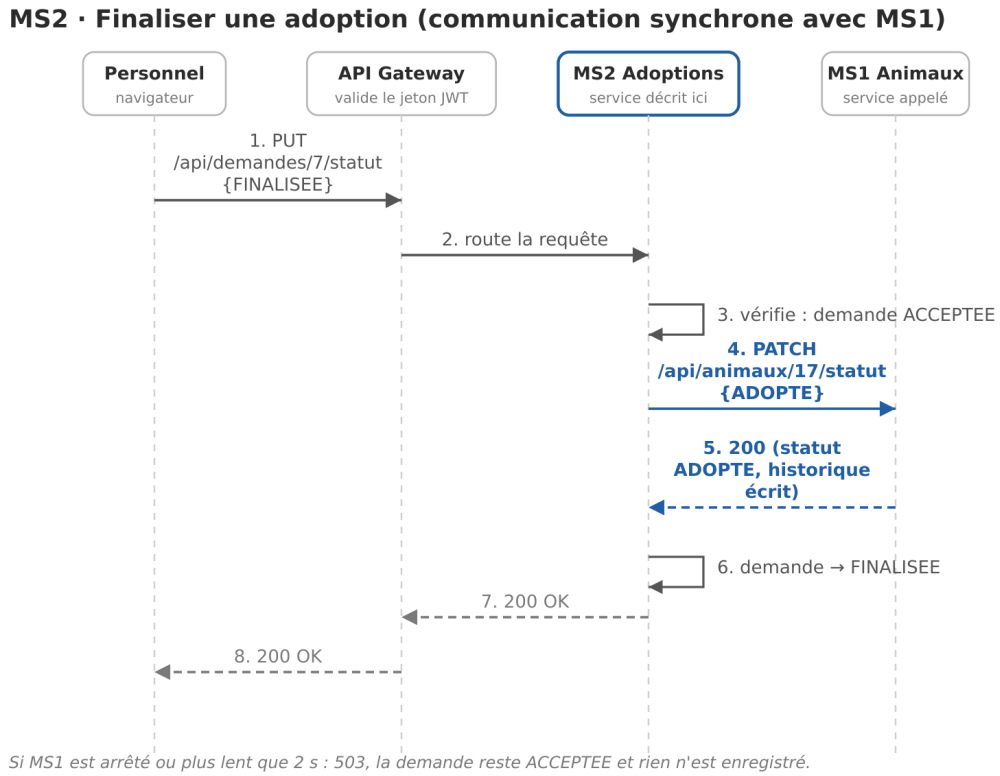


Figure 9 : MS2 · exemple de communication synchrone avec MS1

Communication asynchrone : publier AdoptionFinalisée

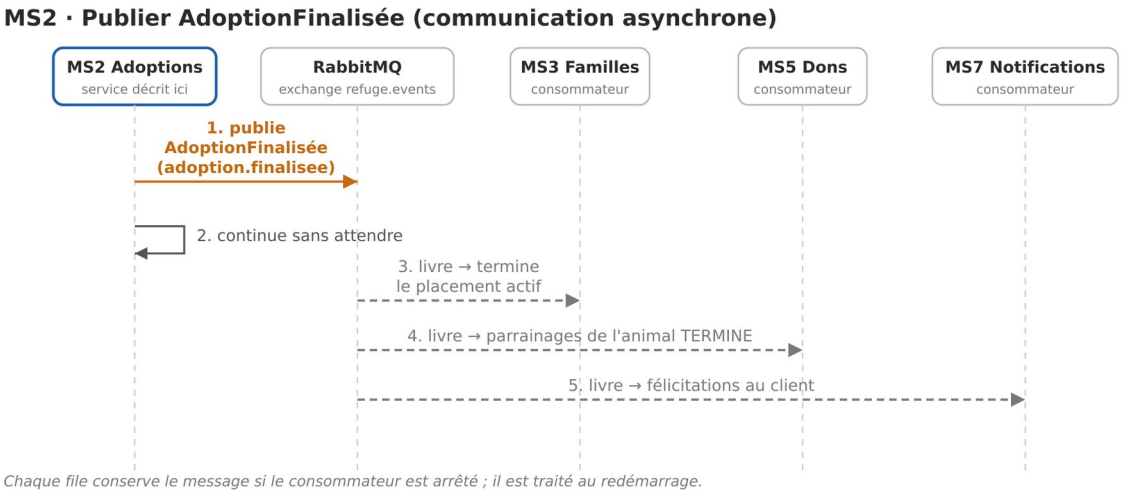


Figure 10 : MS2 · exemple de communication asynchrone

Contrat d'API

Douze endpoints. Codes communs aux endpoints protégés : 401 (non authentifié), 403 (rôle insuffisant).

Méthode	Endpoint	Description	Rôles	Codes
POST	/api/demandes	Créer une demande d'adoption	CLIENT	201, 400, 404, 409, 503
GET	/api/demandes	Récupérer toutes les demandes	PERSONNEL, ADMIN	200

Méthode	Endpoint	Description	Rôles	Codes
GET	/api/demandes/{id}	Récupérer une demande par identifiant	CLIENT (la sienne), PERSONNEL, ADMIN	200, 404
GET	/api/demandes/client/{clientId}	Récupérer les demandes d'un client	CLIENT (les siennes), PERSONNEL, ADMIN	200
PUT	/api/demandes/{id}/statut	Accepter, refuser ou finaliser une demande	PERSONNEL	200, 400, 404, 409, 503
DELETE	/api/demandes/{id}	Annuler une demande	CLIENT (la sienne), PERSONNEL	200, 404, 409
POST	/api/profils	Créer un profil candidat	CLIENT	201, 400
GET	/api/profils/{id}	Consulter un profil	CLIENT (le sien), PERSONNEL	200, 404
PUT	/api/profils/{id}	Modifier un profil	CLIENT (le sien)	200, 400, 404
GET	/api/profils/client/{clientId}	Consulter le profil d'un client	CLIENT (le sien), PERSONNEL	200, 404
GET	/api/demandes/{id}/score	Calculer et consulter le score de compatibilité	PERSONNEL, ADMIN	200, 404, 503
GET	/api/demandes/{id}/compatibilite	Détails de compatibilité	PERSONNEL, ADMIN	200, 404

Appels vers MS1 : `GET /api/animaux/{id}` (existence, disponibilité et caractéristiques de l'animal) et `PATCH /api/animaux/{id}/statut` (passage à ADOPTE après adoption).

Événement	Clé de routage	Déclencheur	Contenu de data
DemandeAdoptionCréée	demande.creee	Dépôt d'une demande	demandeId, clientId, animalId
DemandeAdoptionAcceptée	demande.acceptee	Demande acceptée	demandeId, clientId, animalId
DemandeAdoptionRefusée	demande.refusee	Demande refusée	demandeId, clientId, animalId, motifRefus
AdoptionFinalisée	adoption.finalisee	Adoption finalisée (après la réponse de MS1)	demandeId, clientId, animalId

6.3 MS3 · Familles d'accueil

Rôle

MS3 gère les familles d'accueil du refuge, leur capacité et le placement temporaire d'un animal chez une famille. Il possède sa propre base MySQL et ne garde que `animalId`.

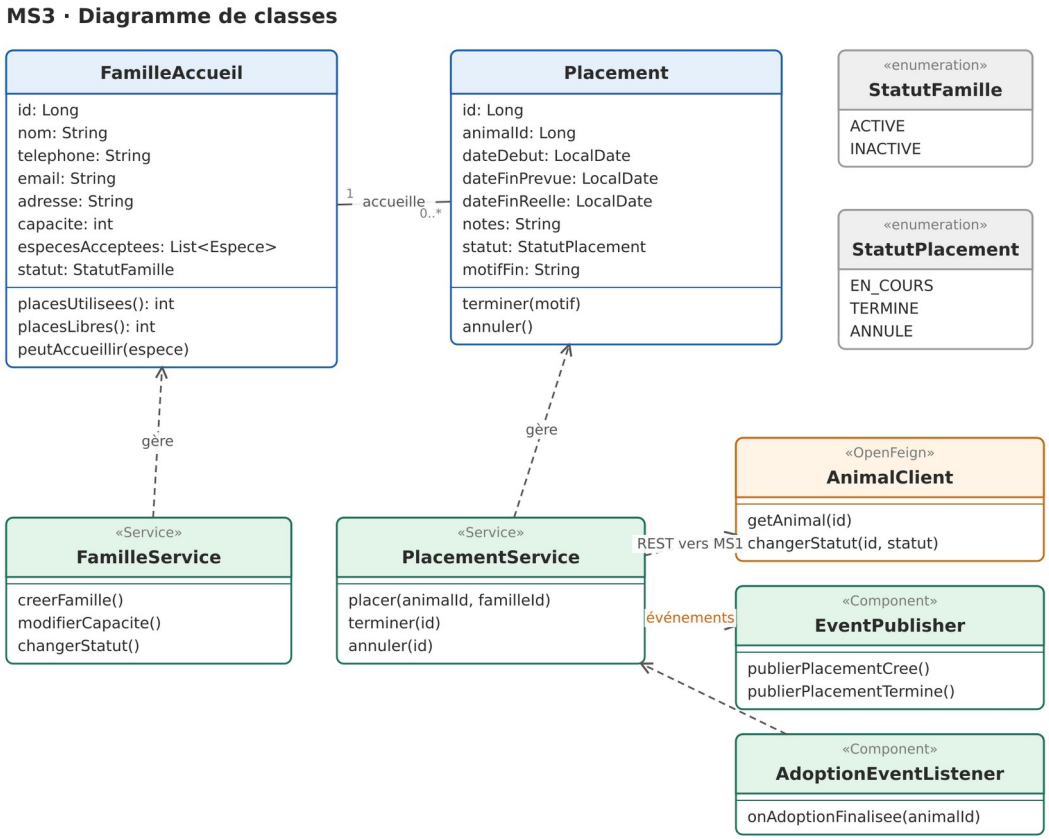
Données (base MySQL)

Entité	Attributs principaux
FamilleAccueil	id, nom, telephone, email, adresse, capacite, especesAcceptees, statut (ACTIVE, INACTIVE)
Placement	id, animalId, familleId, dateDebut, dateFinPrevue, dateFinReelle, notes, motifFin, statut (EN_COURS, TERMINE, ANNULE)

Règles métier

- Places utilisées = nombre de placements EN_COURS de la famille ; elles ne dépassent jamais la capacité.
- Seule une famille ACTIVE peut recevoir un animal, et seulement une espèce qu'elle accepte.
- Seul un animal DISPONIBLE (statut lu dans MS1) peut être placé ; MS3 passe alors l'animal à EN_ACCUEIL dans MS1.
- La capacité ne peut pas être inférieure au nombre de placements en cours (409). Une famille avec des placements en cours ne peut pas être désactivée (409).
- Seul un placement EN_COURS peut être terminé ou annulé. À la fin d'un placement par retour au refuge, l'animal repasse à DISPONIBLE dans MS1.
- Quand MS3 reçoit AdoptionFinalisée pour un animal en accueil, il termine le placement (motif ADOPTION).

Diagramme de classes



Cases grises : énumérations qui typent les attributs.
animalId est une simple référence vers MS1 (aucune copie de la fiche).

Figure 11 : MS3 · diagramme de classes

Communication synchrone : placer un animal

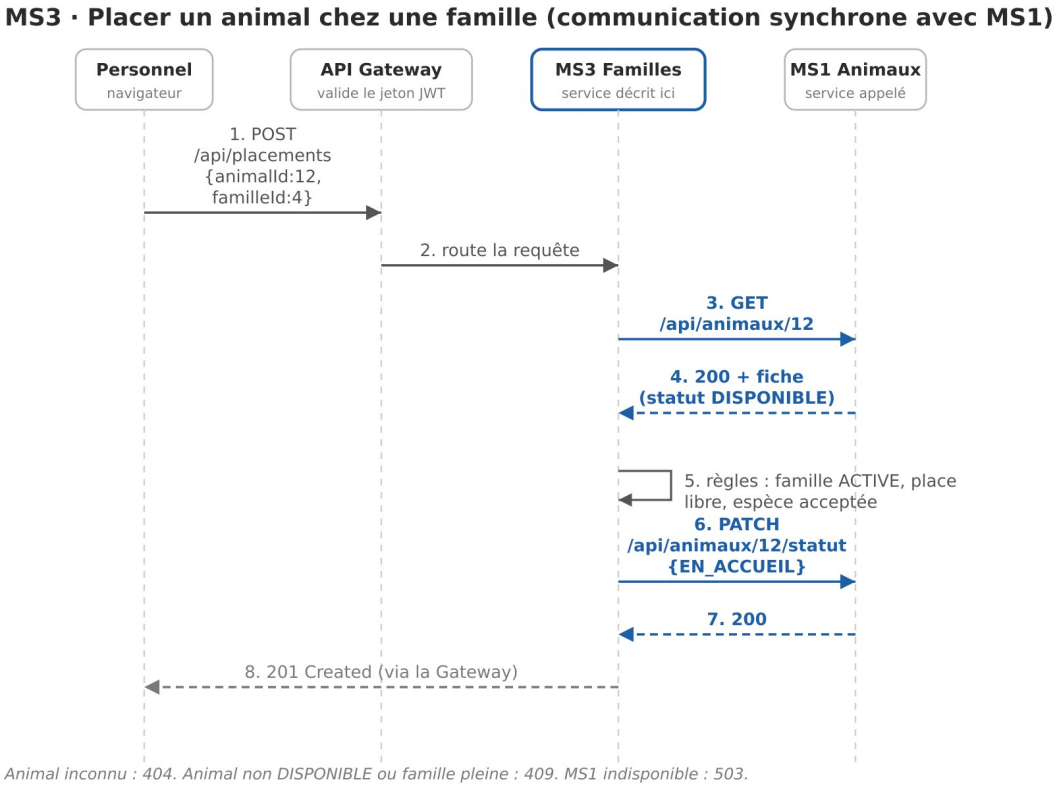


Figure 12 : MS3 · exemple de communication synchrone avec MS1

Communication asynchrone : fin d'un placement

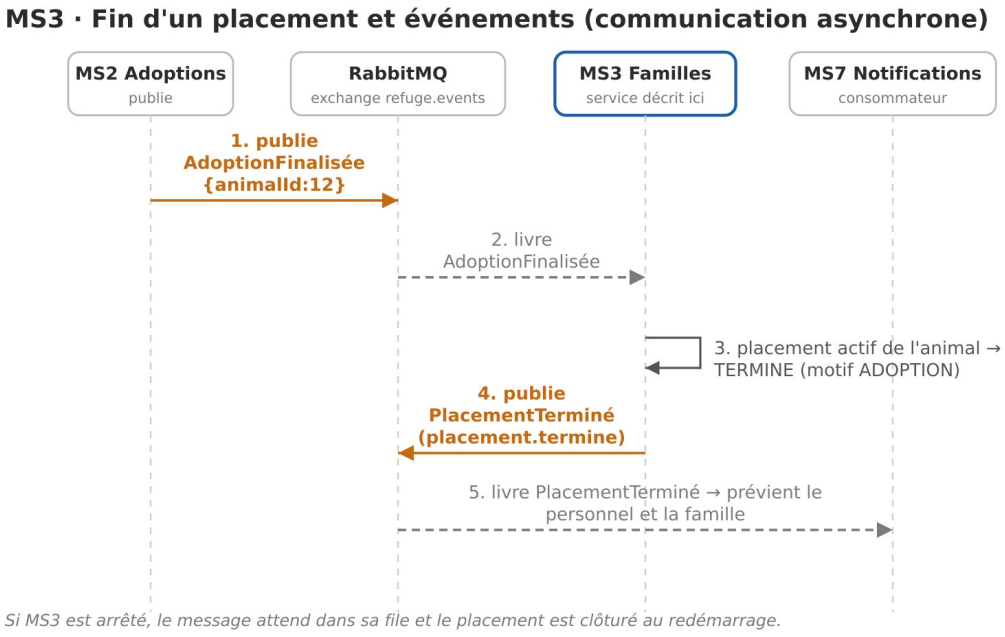


Figure 13 : MS3 · exemple de communication asynchrone

Contrat d'API

Onze endpoints, réservés à PERSONNEL et ADMIN. Codes communs aux endpoints protégés : 401 (non authentifié), 403 (rôle insuffisant).

Méthode	Endpoint	Description	Rôles	Codes
POST	/api/familles	Enregistrer une famille (nom, contact, adresse, capacité, espèces acceptées)	PERSONNEL	201, 400, 409
GET	/api/familles?statut=&espece=&disponibles=&page=&size=	Lister les familles	PERSONNEL	200
GET	/api/familles/{id}	Voir une famille avec ses places utilisées et libres	PERSONNEL	200, 404
PUT	/api/familles/{id}	Modifier les informations ou la capacité	PERSONNEL	200, 400, 404, 409
PATCH	/api/familles/{id}/statut	Activer ou désactiver une famille	PERSONNEL	200, 404, 409
POST	/api/placements	Placer un animal chez une famille	PERSONNEL	201, 400, 404, 409, 503
GET	/api/placements?statut=&familleId=&animalId=&page=&size=	Lister les placements	PERSONNEL	200
GET	/api/placements/{id}	Voir un placement	PERSONNEL	200, 404
PATCH	/api/placements/{id}/terminer	Terminer un placement (retour au refuge)	PERSONNEL	200, 404, 409, 503
PATCH	/api/placements/{id}/annuler	Annuler un placement fait par erreur	PERSONNEL	200, 404, 409, 503
GET	/api/familles/{id}/placements	Placements en cours et passés d'une famille	PERSONNEL	200, 404

Détail de `POST /api/placements` : 409 si la famille est pleine ou inactive, si l'espèce n'est pas acceptée ou si l'animal n'est pas disponible ; 404 si l'animal ou la famille est introuvable ; 503 si MS1 ne répond pas. Corps de la requête :

```
{ "animalId": 12, "familleId": 4, "dateDebut": "2026-11-15", "dateFinPrevue": "2027-01-15",
  "notes": "Besoin d'un traitement deux fois par jour" }
```

Événement	Clé de routage	Déclencheur	Contenu de data
PlacementCréé	placement.cree	Création d'un placement	placementId, animalId, familleId, familleEmail, dateDebut, dateFinPrevue
PlacementTerminé	placement.termine	Fin ou annulation d'un placement	placementId, animalId, familleId, familleEmail, motif (RETOUR_REFUGE, ADOPTION, ANNULATION)

Événement reçu : `AdoptionFinalisée` (de MS2), qui termine le placement en cours de l'animal adopté.

6.4 MS4 · Suivi vétérinaire

Rôle

MS4 gère le suivi médical des animaux : dossiers médicaux, consultations, vaccinations avec rappels et traitements. Il possède sa propre base MySQL et se réfère à l'animal par `animalId` uniquement. Les visites du public et les événements restent dans MS6.

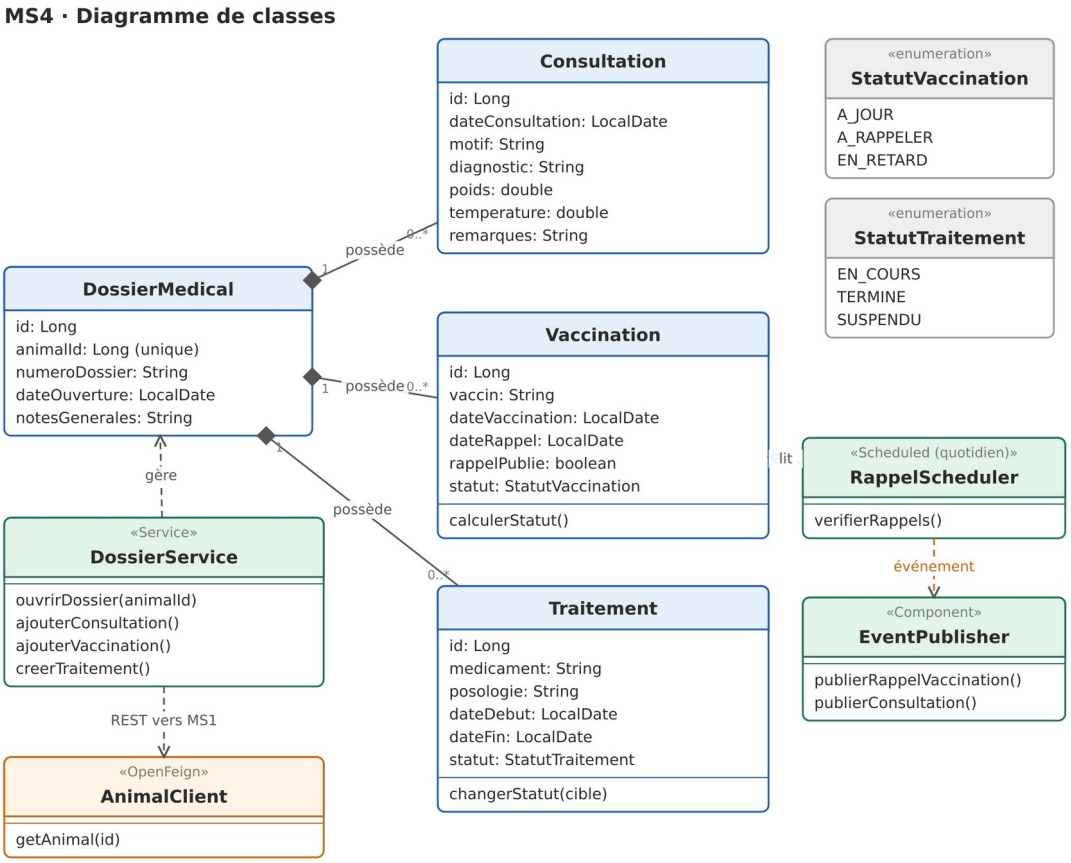
Données (base MySQL)

Entité	Attributs principaux
DossierMedical	id, animalId (unique), numeroDossier, dateOuverture, notesGenerales
Consultation	id, dossierId, dateConsultation, motif, diagnostic, poids, temperature, remarques
Vaccination	id, dossierId, vaccin, dateVaccination, dateRappel, rappelPublie, statut (A_JOUR, A_RAPPELER, EN_RETARD)
Traitement	id, dossierId, medicament, posologie, dateDebut, dateFin, statut (EN_COURS, TERMINE, SUSPENDU)

Règles métier

- Un animal possède au plus un dossier médical (unicité de `animalId`, 409 sinon). À l'ouverture, MS4 vérifie l'existence de l'animal dans MS1 ; aucune table de MS1 n'est copiée.
- Toute consultation, vaccination et tout traitement appartient à un dossier existant.
- La date de rappel est postérieure à la date de vaccination ; la date de fin d'un traitement n'est pas antérieure à son début. Un traitement passe de EN_COURS à TERMINE ou SUSPENDU ; toute autre transition renvoie 409.
- Le statut d'un vaccin est calculé à partir de `dateRappel`.
- Écriture réservée aux membres du personnel du groupe Keycloak « veterinaire » et à l'ADMIN ; les autres membres du personnel consultent seulement.
- Une tâche planifiée repère chaque jour les rappels à venir (sous 7 jours par exemple) et publie un événement une seule fois par rappel ; `GET /vaccinations/reminders` ne publie rien.

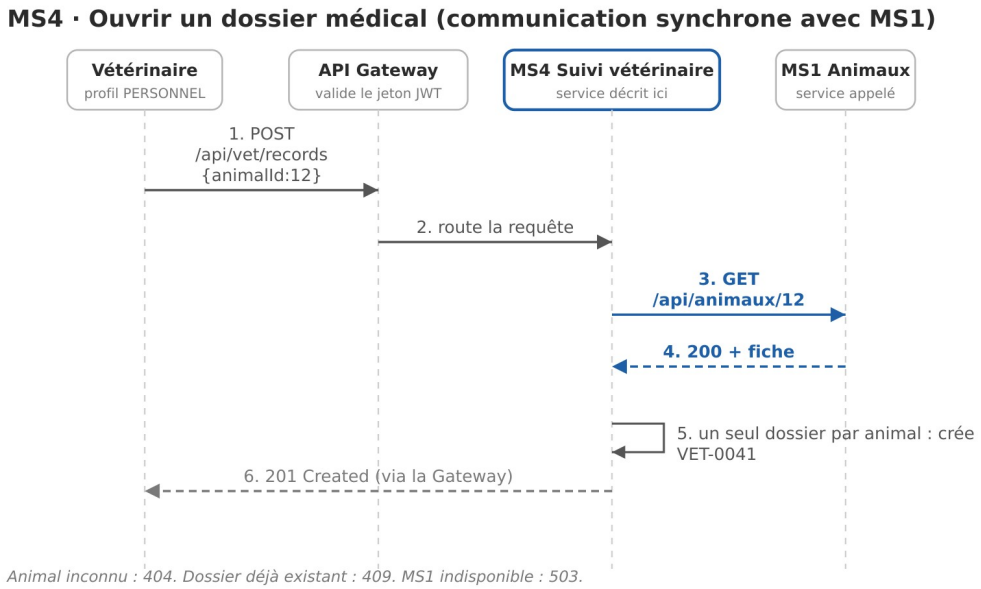
Diagramme de classes



Cases grises : énumérations qui typent les attributs.
animalId est une simple référence vers MS1 (aucune table de MS1 copiée).

Figure 14 : MS4 · diagramme de classes

Communication synchrone : ouvrir un dossier



Animal inconnu : 404. Dossier déjà existant : 409. MS1 indisponible : 503.

Figure 15 : MS4 · exemple de communication synchrone avec MS1

Communication asynchrone : rappel de vaccin

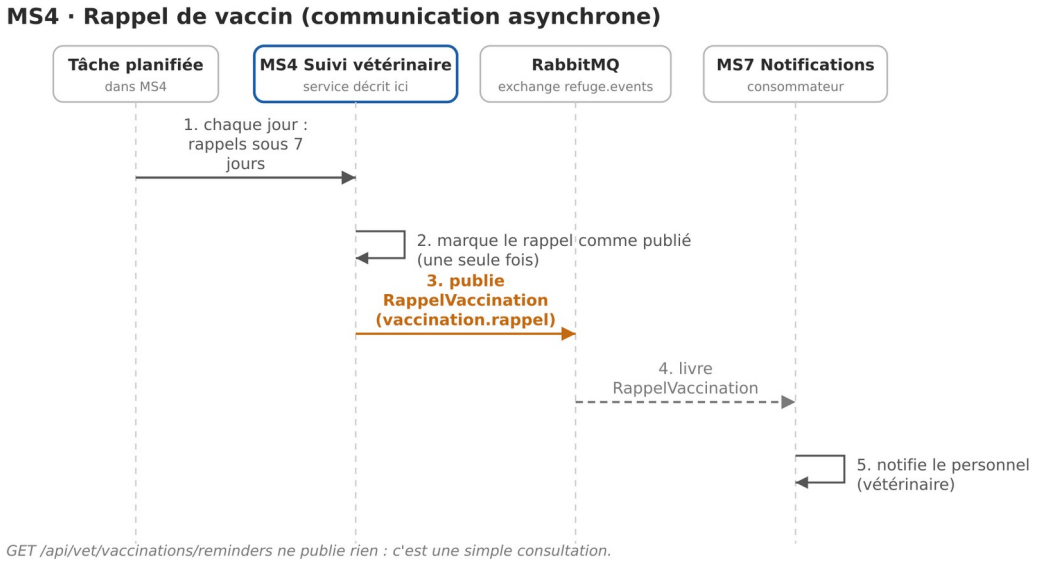


Figure 16 : MS4 · exemple de communication asynchrone

Contrat d'API

Quatorze endpoints sous le préfixe `/api/vet`. Lecture : PERSONNEL et ADMIN ; écriture : PERSONNEL du groupe « veterinaire » et ADMIN. Codes communs aux endpoints protégés : 401 (non authentifié), 403 (rôle insuffisant).

Méthode	Endpoint	Description	Rôles	Codes
POST	/api/vet/records	Ouvrir un dossier pour un animal (vérifie MS1)	Écriture	201, 400, 404, 409, 503
GET	/api/vet/records/{id}	Afficher un dossier	Lecture	200, 404
GET	/api/vet/records/by-animal/{animalId}	Retrouver le dossier d'un animal	Lecture	200, 404
PATCH	/api/vet/records/{id}	Modifier les notes générales	Écriture	200, 400, 404
POST	/api/vet/consultations	Enregistrer une consultation	Écriture	201, 400, 404
GET	/api/vet/records/{id}/consultations	Historique des consultations	Lecture	200, 404
GET	/api/vet/consultations/{id}	Consulter une consultation	Lecture	200, 404
POST	/api/vet/vaccinations	Enregistrer une vaccination	Écriture	201, 400, 404
GET	/api/vet/records/{id}/vaccinations	Historique des vaccinations	Lecture	200, 404
GET	/api/vet/vaccinations/reminders?beforeDate=&statut=	Rappels à venir ou en retard	Lecture	200, 400
PATCH	/api/vet/vaccinations/{id}	Rectifier une vaccination ou son rappel	Écriture	200, 400, 404
POST	/api/vet/treatments	Créer un traitement	Écriture	201, 400, 404
GET	/api/vet/records/{id}/treatments	Lister les traitements d'un dossier	Lecture	200, 404
PATCH	/api/vet/treatments/{id}/status	Changer le statut d'un traitement	Écriture	200, 400, 404, 409

Exemple : `POST /api/vet/vaccinations` avec `{ "dossierId": 41, "vaccin": "Vaccin antirabique", "dateVaccination": "2026-11-11", "dateRappel": "2027-11-11" }` répond 201 avec le statut `A_JOUR`.

Événement	Clé de routage	Déclencheur	Contenu de data
RappelVaccination	vaccination.rappel	Rappel de vaccin à venir (une seule fois par rappel)	animalId, vaccin, dateRappel
ConsultationEnregistrée	consultation.enregistrée	Nouvelle consultation	animalId, consultationId, date

6.5 MS5 · Dons et parrainages

Rôle

MS5 gère les dons faits au refuge et les parrainages mensuels d'animaux. Il possède sa propre base MySQL ; il ne gère pas les fiches animaux (MS1) et garde seulement `animalId`.

- Enregistrer les dons ponctuels, pour le refuge en général ou pour un animal précis, et générer un reçu pour chaque don validé.
- Gérer les parrainages mensuels (création, suspension, reprise, fin).
- Gérer les donateurs et parrains, avec l'historique de leurs dons et parrainages.
- Fournir des statistiques sur les sommes collectées.

Données (base MySQL)

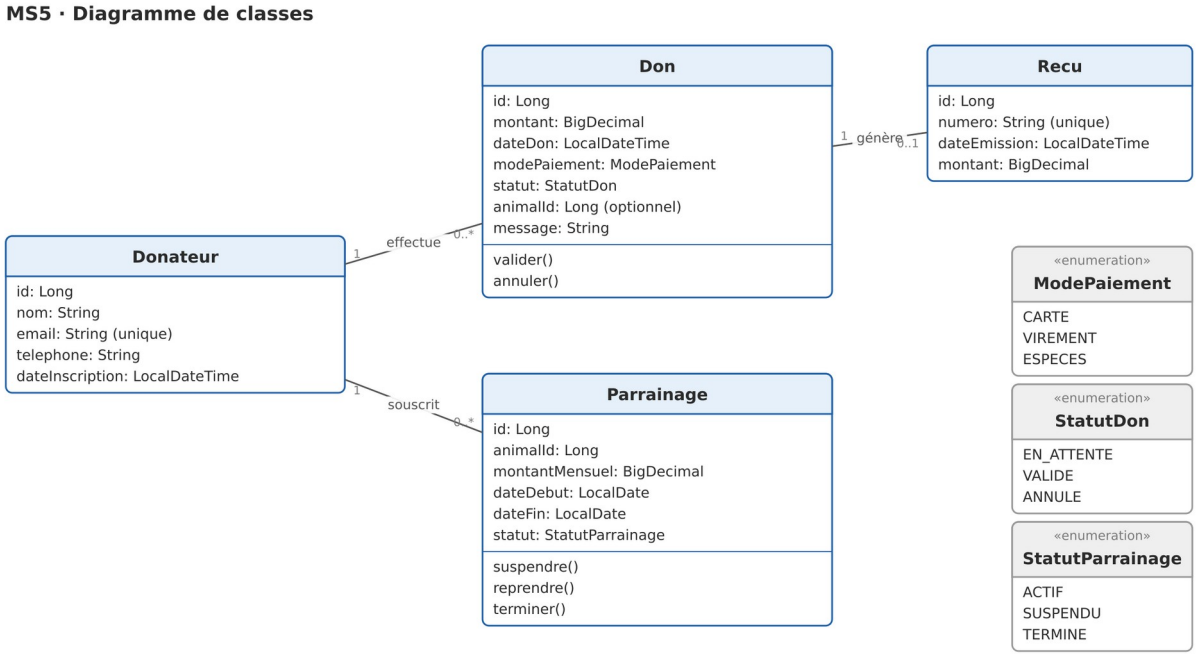
Table	Colonnes principales
donateur	id, nom, email (unique), telephone, date_inscription
don	id, donateur_id (FK), animal_id, montant, mode_paiement, statut, date_don, message
parrainage	id, donateur_id (FK), animal_id, montant_mensuel, date_debut, date_fin, statut
recu	id, don_id (FK, unique), numero (unique), date_emission, montant

`animal_id` n'a pas de clé étrangère : la table des animaux est dans la base de MS1. Énumérations : ModePaiement (CARTE, VIREMENT, ESPECES), StatutDon (EN_ATTENTE, VALIDE, ANNULE), StatutParrainage (ACTIF, SUSPENDU, TERMINE).

Règles métier

- Un don pour un animal exige que l'animal existe (vérifié dans MS1) ; MS1 arrêté : 503.
- Un don validé génère un reçu avec un numéro unique et publie DonValidé.
- Un animal ne peut avoir qu'un parrainage actif par donateur ; un parrainage actif déjà existant renvoie 409. Les transitions de statut interdites renvoient 409.
- Quand MS5 reçoit AdoptionFinalisée, les parrainages actifs de l'animal passent à TERMINE et ParrainageTerminé est publié.
- Un client ne voit que ses propres dons et parrainages.

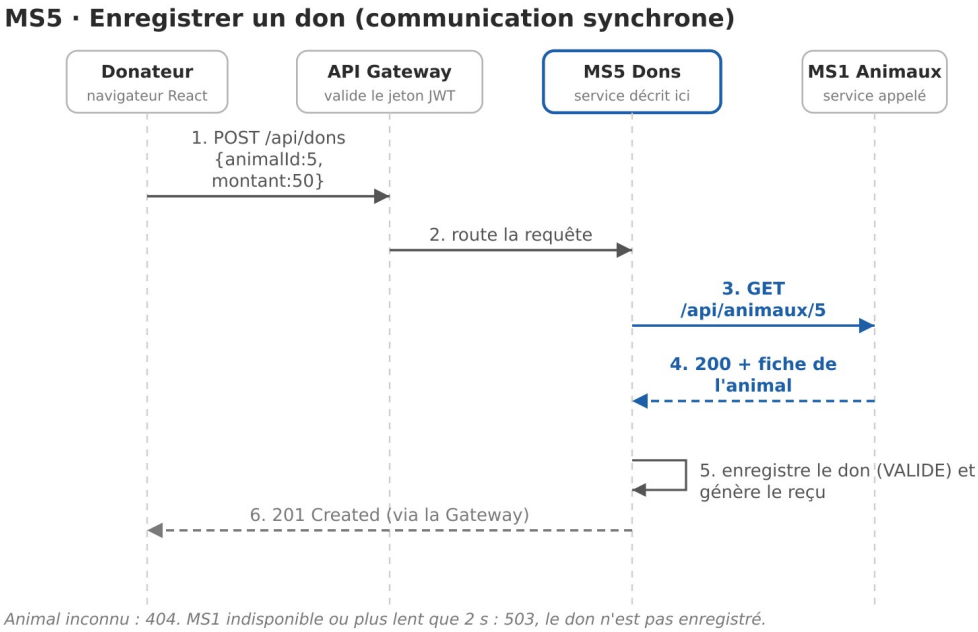
Diagramme de classes



Cases grises : énumérations qui typent les attributs.
animalId est une simple référence vers MS1 : MS5 ne copie pas la fiche de l'animal.

Figure 17 : MS5 · diagramme de classes

Communication synchrone : enregistrer un don pour un animal



Animal inconnu : 404. MS1 indisponible ou plus lent que 2 s : 503, le don n'est pas enregistré.

Figure 18 : MS5 · exemple de communication synchrone avec MS1

Communications asynchrones

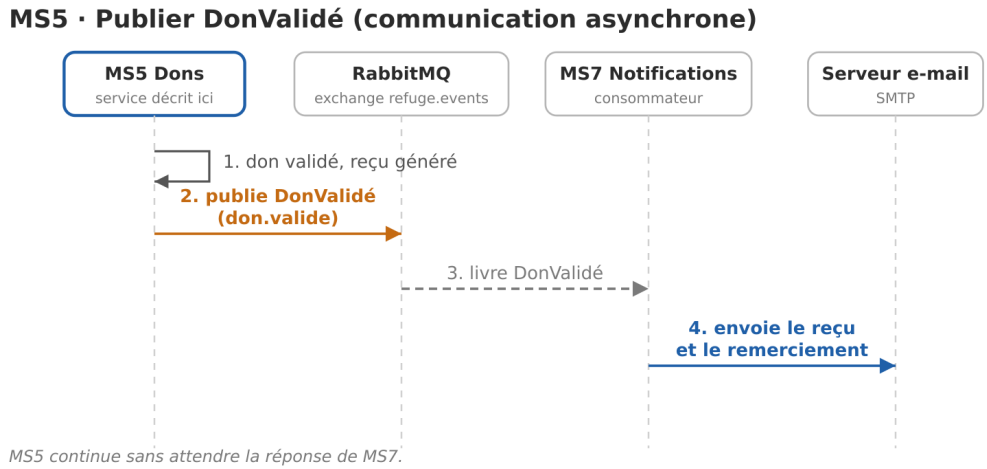


Figure 19 : MS5 · publication de DonValidé et notification du donateur

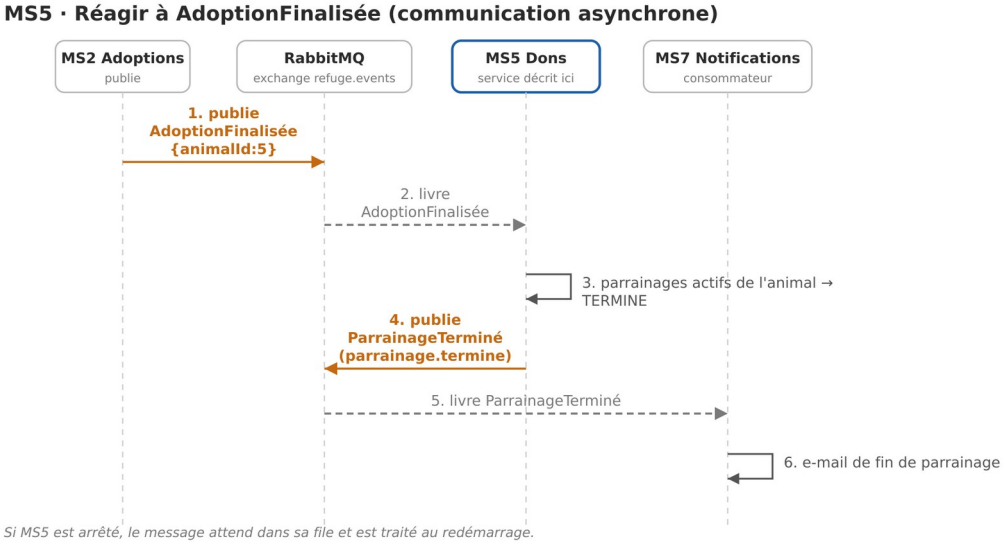


Figure 20 : MS5 · clôture des parrainages après AdoptionFinalisée

Contrat d'API

Quinze endpoints. « (propre) » signifie que le client n'accède qu'à ses propres données. Codes communs aux endpoints protégés : 401 (non authentifié), 403 (rôle insuffisant).

Méthode	Endpoint	Description	Rôles	Codes
POST	/api/donateurs	Créer un donateur	CLIENT, PERSONNEL, ADMIN	201, 400, 409
GET	/api/donateurs/{id}	Détail d'un donateur	CLIENT (propre), PERSONNEL, ADMIN	200, 404
PUT	/api/donateurs/{id}	Modifier un donateur	CLIENT (propre), ADMIN	200, 400, 404
GET	/api/donateurs/{id}/dons	Historique des dons d'un donateur	CLIENT (propre), PERSONNEL, ADMIN	200, 404
POST	/api/dons	Enregistrer un don (génère un reçu, publie DonValidé)	CLIENT, ADMIN	201, 400, 404, 503
GET	/api/dons?animalId=&donateurId=	Lister et filtrer les dons	PERSONNEL, ADMIN	200, 400

Méthode	Endpoint	Description	Rôles	Codes
GET	/api/dons/{id}	Détail d'un don	CLIENT (propre), PERSONNEL, ADMIN	200, 404
GET	/api/dons/{id}/recu	Obtenir le reçu d'un don	CLIENT (propre), PERSONNEL, ADMIN	200, 404
PATCH	/api/dons/{id}/annuler	Annuler un don	ADMIN	200, 404, 409
POST	/api/parrainages	Créer un parrainage (publie ParrainageCréé)	CLIENT	201, 400, 404, 409, 503
GET	/api/parrainages?animalId=&statut=	Lister et filtrer les parrainages	PERSONNEL, ADMIN	200, 400
GET	/api/parrainages/{id}	Détail d'un parrainage	CLIENT (propre), PERSONNEL, ADMIN	200, 404
PATCH	/api/parrainages/{id}/statut	Suspendre, reprendre ou terminer (publie ParrainageTerminé si terminé)	CLIENT (propre), ADMIN	200, 400, 404, 409
GET	/api/animaux/{animalId}/parrains	Lister les parrains d'un animal	PERSONNEL, ADMIN	200, 404
GET	/api/stats/dons?debut=&fin=	Total collecté sur une période et animaux les plus soutenus	PERSONNEL, ADMIN	200, 400

Événement	Clé de routage	Déclencheur	Contenu de data
DonValidé	don.valide	Don validé et reçu généré	donId, donateurId, donateurEmail, animalId (optionnel), montant, numeroRecu
ParrainageCréé	parrainage.cree	Création d'un parrainage	parrainageId, donateurId, donateurEmail, animalId, montantMensuel
ParrainageTerminé	parrainage.termine	Fin d'un parrainage (manuelle ou automatique)	parrainageId, donateurId, donateurEmail, animalId, motif

Événement reçu : `AdoptionFinalisée` (de MS2), qui contient au minimum `animalId`.

6.6 MS6 · Visites et rendez-vous

Rôle

MS6 gère les **visites** des animaux au refuge (créneaux et réservations) et les **événements** (journées d'adoption, ateliers) avec leurs inscriptions et leur liste d'attente. Un refuge doit faire venir du monde pour trouver des adoptants, des donateurs et des bénévoles. Une visite est individuelle (une personne voit un animal sur un créneau) ; un événement est collectif (date, lieu, places limitées). Les deux reposent sur la même logique de places limitées.

Données (base MySQL)

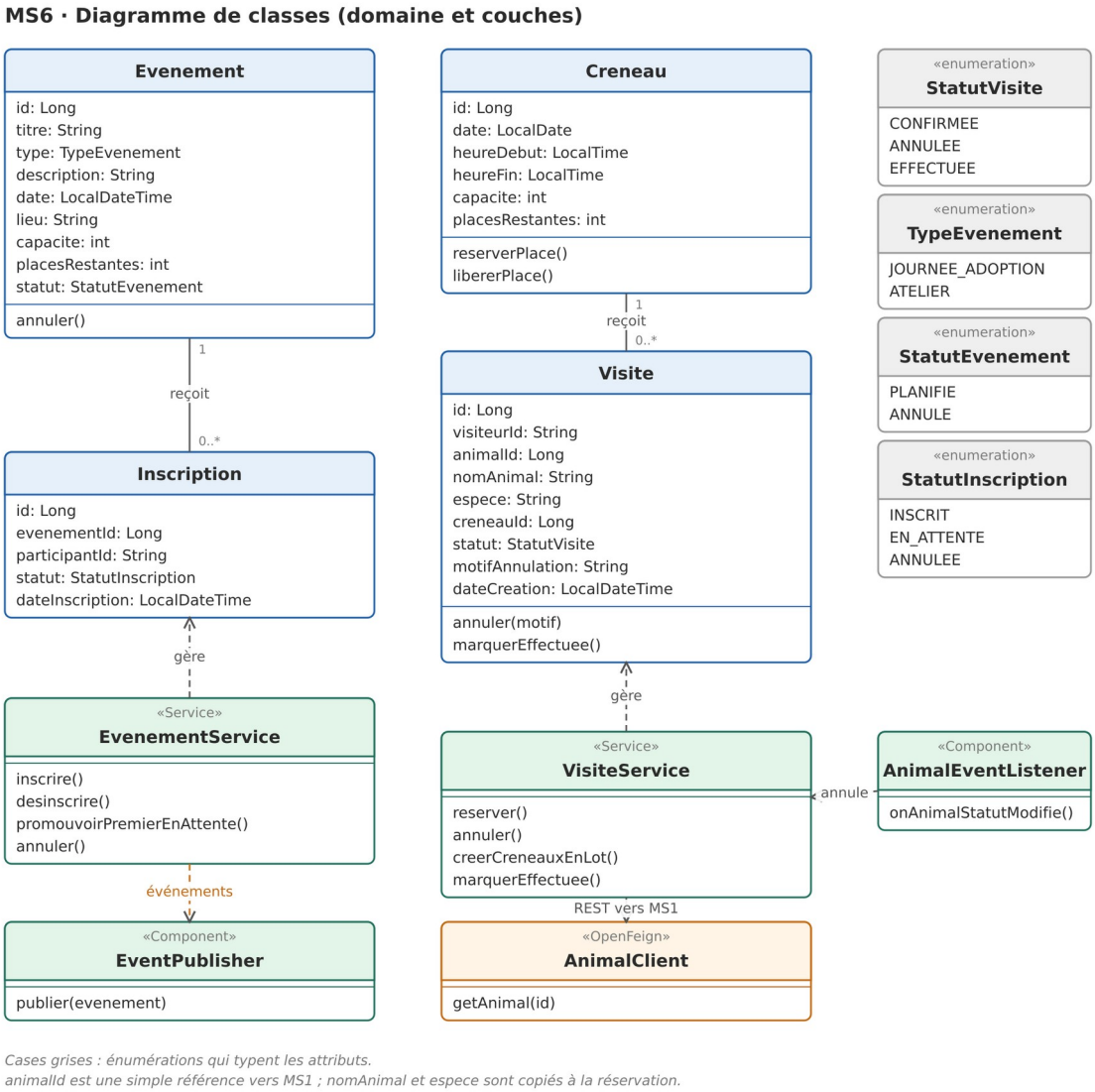
Entité	Attributs principaux
Creneau	id, date, heureDebut, heureFin, capacite, placesRestantes
Visite	id, visiteurId, animalId, nomAnimal, espece, creneauId, statut (CONFIRMEE, ANNULEE, EFFECTUEE), motifAnnulation, dateCreation
Evenement	id, titre, type (JOURNEE_ADOPTION, ATELIER), description, date, lieu, capacite, placesRestantes, statut (PLANIFIE, ANNULE)
Inscription	id, evenementId, participantId, statut (INSCRIT, EN_ATTENTE, ANNULEE), dateInscription

`nomAnimal` et `espece` sont copiés depuis MS1 au moment de la réservation ; ils ne sont pas mis à jour ensuite.

Règles métier

- **Confirmation immédiate** : une visite est confirmée tout de suite, sans validation du personnel.
- **Annulation** : un visiteur annule jusqu'à 24 h avant le créneau (409 sinon) et seulement ses propres visites (403 sinon). Le personnel annule à tout moment.
- **Limites** : au plus 3 visites par animal et par jour (paramétrable) et une seule visite par visiteur, animal et jour (409 sinon).
- **Capacité** : la place d'un créneau est décrémentée de façon atomique ; créneau complet : 409 ; créneau passé : 400.
- **Animal visitable** : seuls les animaux DISPONIBLE (statut lu dans MS1) peuvent être visités, sinon 409.
- **Créneaux** : le personnel les crée en lot (série), les modifie et les supprime. Si la date ou l'heure change, ou si le créneau est supprimé, les visites réservées sont annulées et les visiteurs prévenus. La capacité ne peut pas descendre sous le nombre de visites confirmées (409).
- **Animal plus disponible** : à la réception de StatutAnimalModifié (de MS1), si le nouveau statut n'est pas DISPONIBLE (adoption, accueil, soins, réservation), les visites à venir de l'animal sont annulées et les visiteurs prévenus.
- **Inscription à un événement** : s'il reste des places, statut INSCRIT ; sinon la personne passe en liste d'attente (EN_ATTENTE). Une seule inscription par personne et par événement (409). Événement annulé : 409.
- **Désinscription** : elle libère la place et la première personne en attente est promue automatiquement (INSCRIT) et prévenue.
- **Annulation d'un événement** : l'ADMIN peut annuler un événement ; toutes les inscriptions passent à ANNULEE et les inscrits sont prévenus (409 si déjà annulé ou passé).

Diagramme de classes



Cases grises : énumérations qui typent les attributs.
animalId est une simple référence vers MS1 ; nomAnimal et espece sont copiés à la réservation.

Figure 21 : MS6 · diagramme de classes (domaine et couches)

Cas d'utilisation

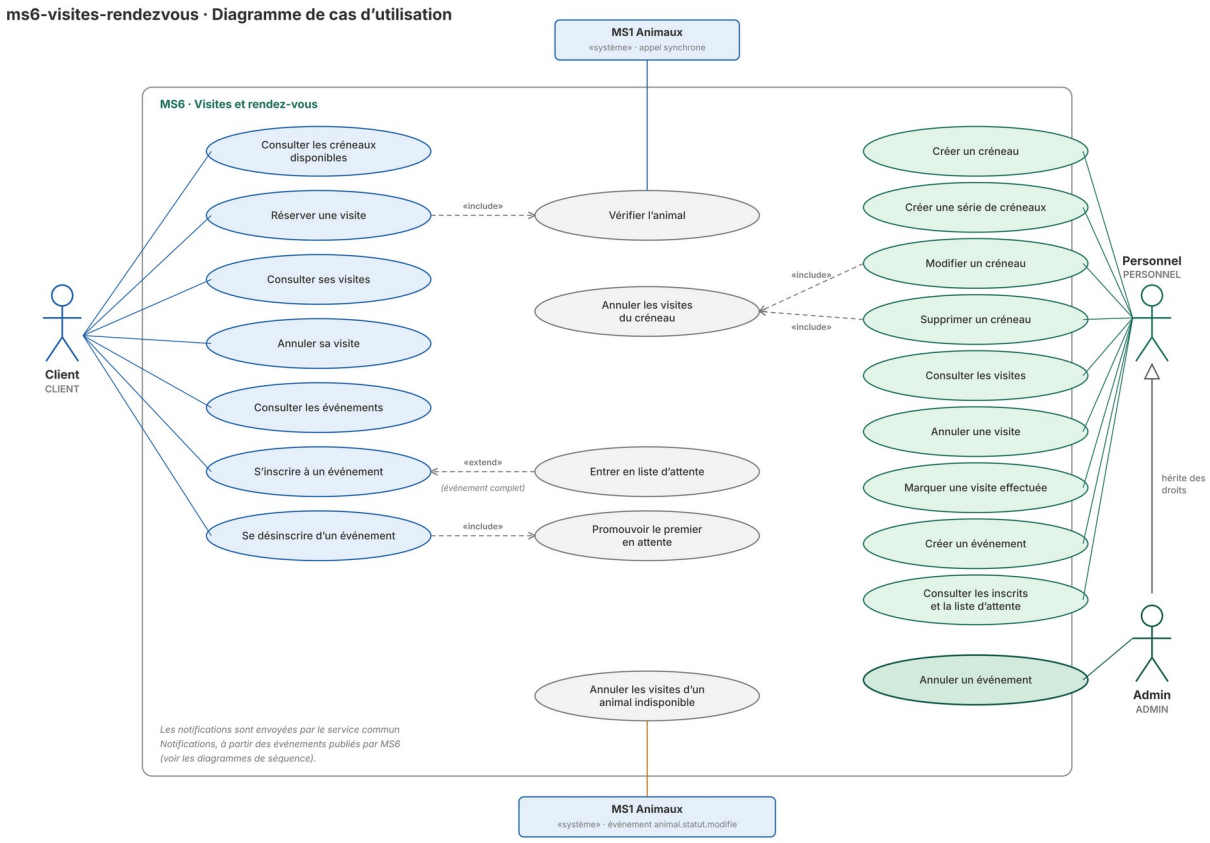


Figure 22 : MS6 · cas d'utilisation

Communication synchrone : réserver une visite

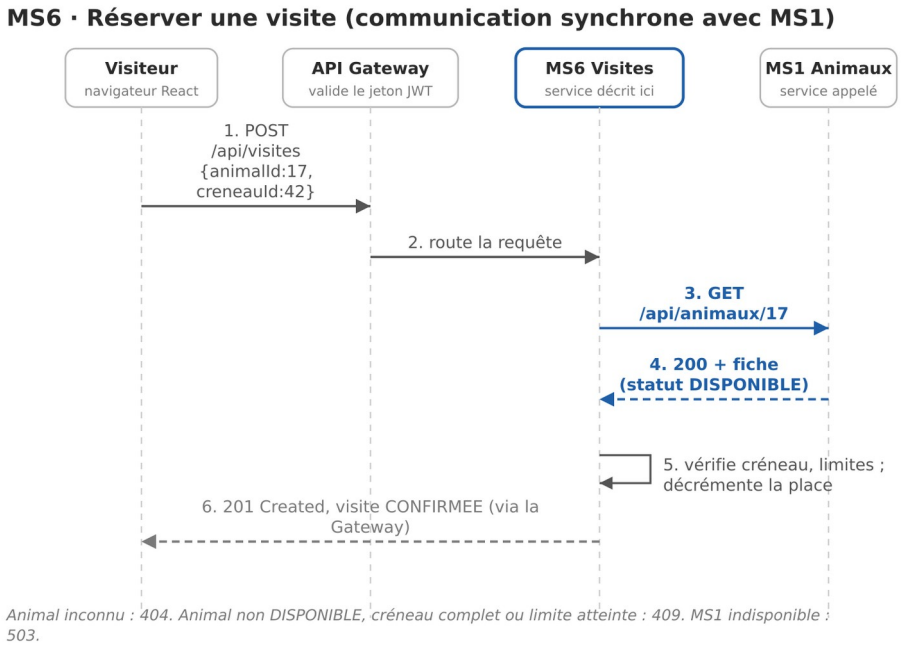


Figure 23 : MS6 · exemple de communication synchrone avec MS1

Communications asynchrones

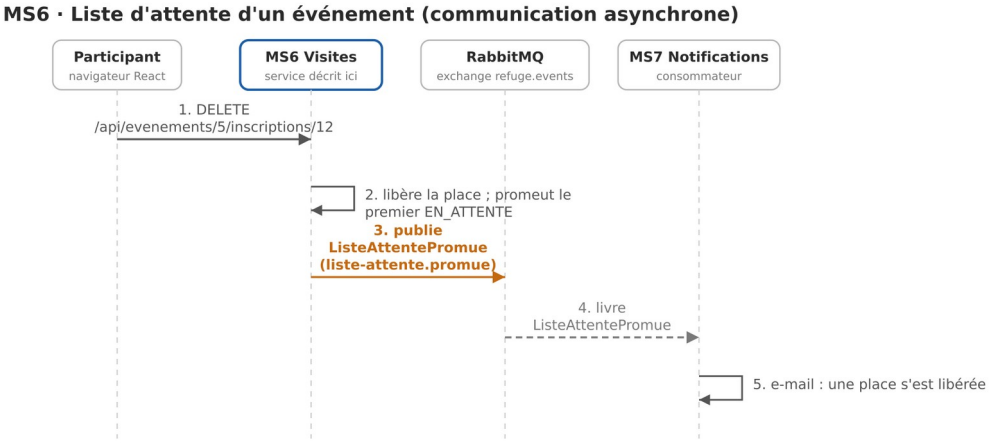


Figure 24 : MS6 · promotion de la liste d'attente

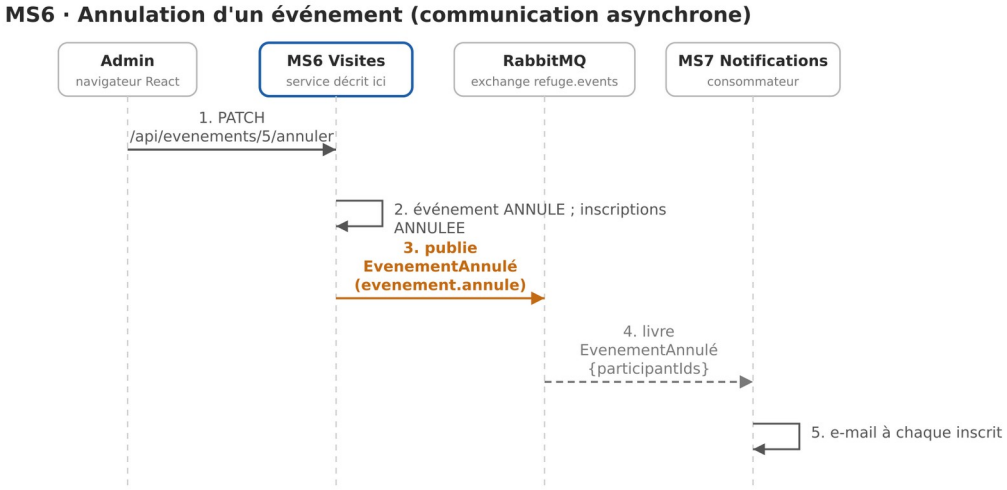


Figure 25 : MS6 · annulation d'un événement

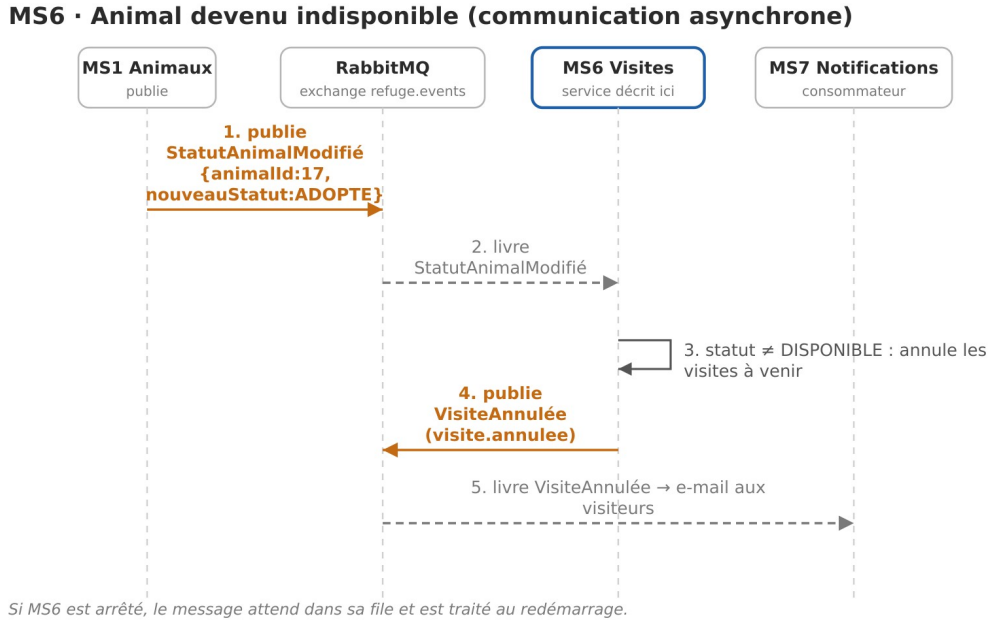


Figure 26 : MS6 · réaction à un changement de statut de l'animal

Contrat d'API

Quinze endpoints sous les préfixes `/api/visites` et `/api/evenements`. Codes communs aux endpoints protégés : 401 (non authentifié), 403 (rôle insuffisant).

Méthode	Endpoint	Description	Rôles	Codes
POST	/api/visites/creneaux	Créer un créneau	PERSONNEL	201, 400
POST	/api/visites/creneaux/lot	Créer une série de créneaux	PERSONNEL	201, 400
GET	/api/visites/creneaux?date=	Lister les créneaux disponibles	Tous	200
PUT	/api/visites/creneaux/{id}	Modifier un créneau (annule les visites si date ou heure change)	PERSONNEL	200, 400, 404, 409
DELETE	/api/visites/creneaux/{id}	Supprimer un créneau (annule ses visites)	PERSONNEL	204, 404
POST	/api/visites	Réserver une visite	CLIENT, PERSONNEL	201, 400, 404, 409, 503
GET	/api/visites?visiteurId=&animalId=	Lister les visites	CLIENT (les siennes), PERSONNEL	200
DELETE	/api/visites/{id}	Annuler une visite (24 h avant maximum pour un visiteur)	CLIENT (la sienne), PERSONNEL	204, 403, 404, 409
PATCH	/api/visites/{id}/effectuee	Marquer une visite comme effectuée	PERSONNEL	200, 404, 409
POST	/api/evenements	Créer un événement	PERSONNEL	201, 400
GET	/api/evenements?aVenir=true	Lister les événements	Tous	200
PATCH	/api/evenements/{id}/annuler	Annuler un événement	ADMIN	200, 404, 409
POST	/api/evenements/{id}/inscriptions	S'inscrire (ou entrer en liste d'attente)	Tous (connectés)	201, 404, 409
DELETE	/api/evenements/{id}/inscriptions/{inscriptionId}	Se désinscrire (promeut le premier en attente)	CLIENT (la sienne), PERSONNEL	204, 403, 404
GET	/api/evenements/{id}/inscriptions	Lister les inscrits et la liste d'attente	PERSONNEL	200, 404

Codes de `POST /api/visites` : 201 visite créée · 400 données invalides ou créneau passé · 404 créneau ou animal inconnu · 409 créneau complet, animal non visitable, limite atteinte ou doublon · 503 MS1 indisponible. Exemple de corps : `{ "animalId": 17, "creneauId": 42 }` (le `visiteurId` vient du jeton) ; réponse : `{ "id": 301, "nomAnimal": "Luna", "espece": "CHAT", "statut": "CONFIRMEE" }`.

Événement	Clé de routage	Déclencheur	Contenu de data
VisiteCréée	visite.creee	Réservation d'une visite	visiteId, visiteurId, animalId, date
VisiteAnnulée	visite.annulee	Annulation d'une visite	visiteId, visiteurId, animalId, motif
EvenementAnnulé	evenement.annule	Annulation d'un événement	evenementId, titre, date, participantIds
ListeAttentePromue	liste-attente.promue	Promotion du premier en attente	evenementId, titre, participantId

Événement reçu : `StatutAnimalModifié` (clé `animal.statut.modifie`, de MS1) : `{ animalId, ancienStatut, nouveauStatut }`. Si le nouveau statut n'est pas DISPONIBLE, MS6 annule les visites à venir de l'animal.

6.7 MS7 · Notifications (service commun)

Rôle

MS7 est le service commun, développé par toute l'équipe. Il écoute les événements publiés par les autres services et en fait des notifications : e-mails (reçu de don, confirmation de visite, décision sur une demande d'adoption, etc.) et notifications internes pour le personnel. Il utilise **MongoDB**, la technologie avancée du projet : chaque notification est un document, et les contenus varient beaucoup d'un événement à l'autre.

Données (collection MongoDB « notifications »)

Champ	Description
id	Identifiant du document
eventId	Identifiant de l'événement reçu (unique, pour l'idempotence)
typeEvenement	Nom de l'événement (DonValidé, VisiteCréée, etc.)
destinataireId / destinataireEmail / role	Le destinataire : un utilisateur (identifiant Keycloak et e-mail) ou un rôle (PERSONNEL)
canal	EMAIL ou INTERNE
titre, message	Contenu construit à partir du gabarit du type d'événement
statut	EN_ATTENTE, ENVOYEE, ECHEC ou LUE
dateCreation, dateEnvoi	Dates de création et d'envoi

Règles métier

- MS7 n'appelle aucun autre service : l'adresse e-mail arrive dans l'événement quand elle est connue (par exemple `donateurEmail`, `familleEmail`), sinon MS7 la demande à Keycloak à partir de l'identifiant de l'utilisateur.
- Un événement déjà traité (même `eventId`) est ignoré.
- Si l'envoi échoue, la notification est enregistrée avec le statut ECHEC ; l'ADMIN peut la renvoyer.
- Un utilisateur ne lit que ses propres notifications ; une notification destinée au personnel est visible par tous les membres du personnel.
- Chaque membre de l'équipe écrit dans MS7 l'écouteur et le gabarit des événements de son propre service.

Événement reçu	Destinataire	Notification
AnimalCréé	PERSONNEL	Nouvel animal enregistré (interne)
StatutAnimalModifié	PERSONNEL	Changement de statut d'un animal (interne)
DemandeAdoptionCréée / Acceptée / Refusée	Client	Accusé de réception, acceptation, refus avec motif (e-mail)
AdoptionFinalisée	Client	Félicitations pour l'adoption (e-mail)
PlacementCréé / PlacementTerminé	Famille, PERSONNEL	Début et fin d'un placement (e-mail à la famille, interne au personnel)
RappelVaccination	PERSONNEL	Vaccin à renouveler avant la date de rappel (interne)
ConsultationEnregistrée	PERSONNEL	Nouvelle consultation (interne)
DonValidé	Donateur	Reçu et remerciement (e-mail)
ParrainageCréé / ParrainageTerminé	Parrain	Confirmation et fin du parrainage (e-mail)
VisiteCréée / VisiteAnnulée	Visiteur	Confirmation et annulation de visite (e-mail)

Événement reçu	Destinataire	Notification
EvenementAnnulé / ListeAttentePromue	Participants / participant	Annulation d'un événement ; place obtenue en liste d'attente (e-mail)

Diagramme de classes

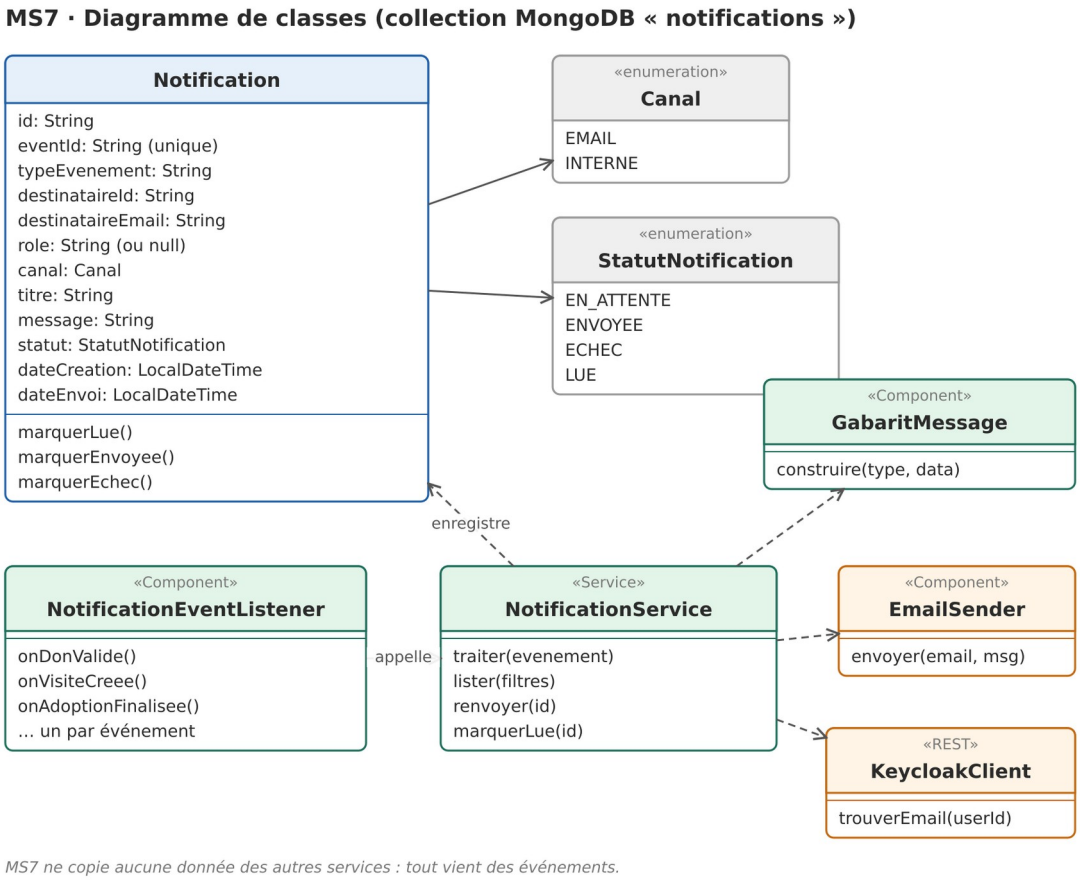


Figure 27 : MS7 · diagramme de classes

Communication asynchrone : de l'événement à l'e-mail

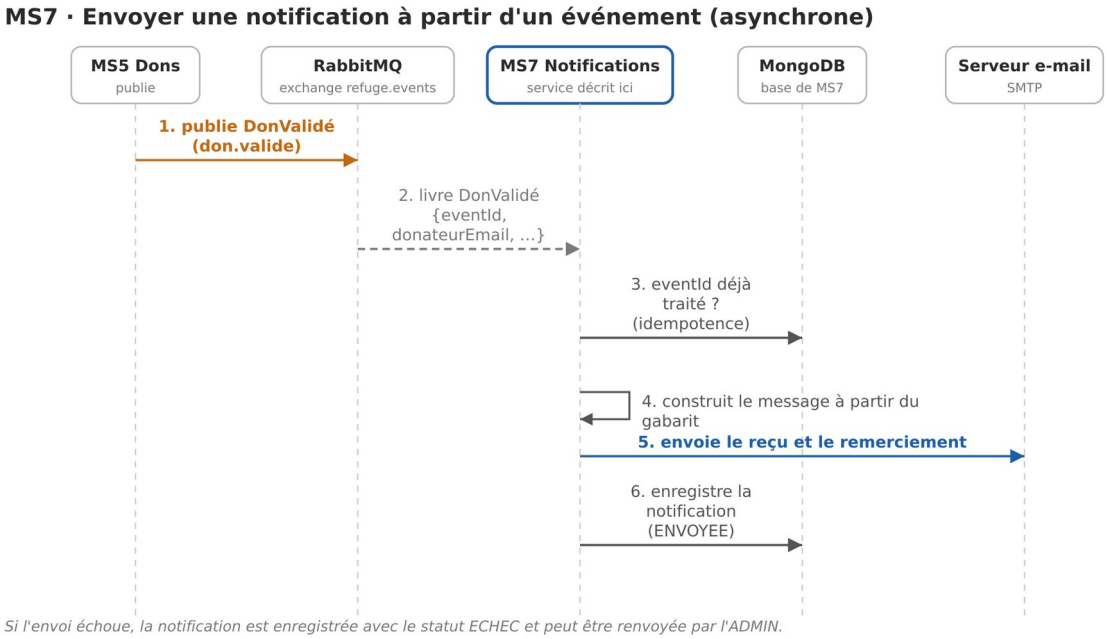


Figure 28 : MS7 · exemple de communication asynchrone

Communication synchrone : consulter ses notifications

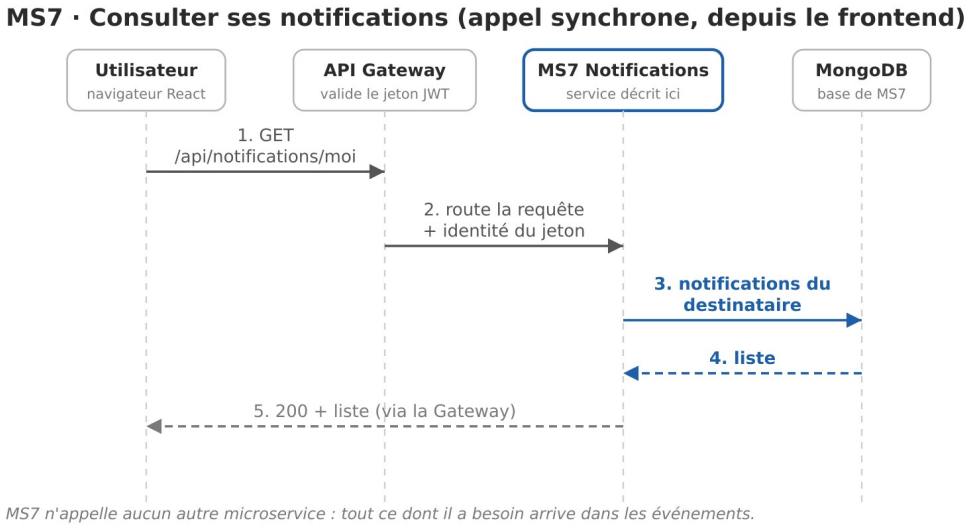


Figure 29 : MS7 · exemple de communication synchrone (appel du frontend)

Contrat d'API

Cinq endpoints sous le préfixe `/api/notifications`. Codes communs aux endpoints protégés : 401 (non authentifié), 403 (rôle insuffisant).

Méthode	Endpoint	Description	Rôles	Codes
GET	/api/notifications/moi?statut=&page=&size=	Mes notifications	CLIENT, PERSONNEL, ADMIN	200
GET	/api/notifications/{id}	Détail d'une notification	Son destinataire, PERSONNEL, ADMIN	200, 403, 404
PATCH	/api/notifications/{id}/lue	Marquer comme lue	Son destinataire	200, 403, 404

Méthode	Endpoint	Description	Rôles	Codes
GET	/api/notifications?type=&statut=&page=&size=	Lister toutes les notifications	ADMIN	200, 400
POST	/api/notifications/{id}/renvoyer	Renvoyer une notification en échec	ADMIN	200, 404, 409

Événements reçus : tous ceux de la section 7.2. MS7 ne publie aucun événement.

7. Interactions entre les services

7.1 Appels synchrones

Tous les appels synchrones vont vers MS1, le service de référence des animaux. L'appelant transmet le jeton de l'utilisateur ; le délai maximal est de 2 secondes, après quoi l'appelant répond 503 sans rien enregistrer.

Appelant	Appel vers MS1	Raison	Réponses prévues
MS2 Demandes	`GET /api/animaux/{id}`	Vérifier que l'animal existe et est disponible ; lire ses caractéristiques pour le score	404, 409, 503
MS2 Demandes	`PATCH /api/animaux/{id}/statut`	Passer l'animal à ADOPTE à la finalisation	404, 409, 503
MS3 Familles	`GET /api/animaux/{id}`	Vérifier que l'animal est DISPONIBLE avant un placement	404, 409, 503
MS3 Familles	`PATCH /api/animaux/{id}/statut`	Passer à EN_ACCUEIL au placement, puis à DISPONIBLE au retour au refuge	404, 409, 503
MS4 Vétérinaire	`GET /api/animaux/{id}`	Vérifier l'existence de l'animal avant d'ouvrir son dossier	404, 503
MS5 Dons	`GET /api/animaux/{id}`	Vérifier l'existence de l'animal avant un don ou un parrainage	404, 503
MS6 Visites	`GET /api/animaux/{id}`	Vérifier que l'animal est DISPONIBLE avant de confirmer une visite	404, 409, 503

7.2 Catalogue des événements

Dix-sept événements circulent sur l'exchange `refuge.events`, avec l'enveloppe commune `eventId`, `eventType`, `occurredAt` et `data`.

Événement	Clé de routage	Publié par	Consommé par	Contenu de data
AnimalCréé	animal.cree	MS1	MS7	animalId, nom, espece, statut
StatutAnimalModifié	animal.statut.modifie	MS1	MS6, MS7	animalId, ancienStatut, nouveauStatut, origine
DemandeAdoptionCréée	demande.creee	MS2	MS7	demandeId, clientId, animalId
DemandeAdoptionAcceptée	demande.acceptee	MS2	MS7	demandeId, clientId, animalId
DemandeAdoptionRefusée	demande.refusee	MS2	MS7	demandeId, clientId, animalId, motifRefus
AdoptionFinalisée	adoption.finalisee	MS2	MS3, MS5, MS7	demandeId, clientId, animalId
PlacementCréé	placement.cree	MS3	MS7	placementId, animalId, familleId, familleEmail, dateDebut, dateFinPrevue
PlacementTerminé	placement.termine	MS3	MS7	placementId, animalId, familleId, familleEmail, motif
RappelVaccination	vaccination.rappel	MS4	MS7	animalId, vaccin, dateRappel
ConsultationEnregistrée	consultation.enregistree	MS4	MS7	animalId, consultationId, date
DonValidé	don.valide	MS5	MS7	donId, donateurId, donateurEmail, animalId, montant, numeroRecu
ParrainageCréé	parrainage.cree	MS5	MS7	parrainageId, donateurId, donateurEmail, animalId, montantMensuel
ParrainageTerminé	parrainage.termine	MS5	MS7	parrainageId, donateurId, donateurEmail, animalId, motif

Événement	Clé de routage	Publié par	Consommé par	Contenu de data
VisiteCréée	visite.creee	MS6	MS7	visiteId, visiteurId, animalId, date
VisiteAnnulée	visite.annulee	MS6	MS7	visiteId, visiteurId, animalId, motif
EvenementAnnulé	evenement.annule	MS6	MS7	evenementId, titre, date, participantIds
ListeAttentePromue	liste-attente.promue	MS6	MS7	evenementId, titre, participantId

Exemple de message (`DonValidé`) :

```
{
  "eventId": "b7e2c1a0-4f1d-4e6a-9c2b-1d2f3a4b5c6d",
  "eventType": "DonValidé",
  "occurredAt": "2026-11-11T10:30:00",
  "data": {
    "donId": 101, "donateurId": 12, "donateurEmail": "ahmed@example.com",
    "animalId": 5, "montant": 50.0, "numeroRecu": "REC-2026-000101"
  }
}
```

7.3 Scénario de bout en bout : adopter un animal

1. Le client consulte le catalogue (`GET /api/animaux`, MS1) et réserve une visite pour l'animal 17 (`POST /api/visites`, MS6). MS6 appelle MS1 pour vérifier que l'animal est DISPONIBLE, puis publie `VisiteCréée` ; MS7 envoie la confirmation.
2. Le client enregistre son profil et dépose une demande (`POST /api/profils`, `POST /api/demandes`, MS2). MS2 appelle MS1 pour vérifier l'animal et publie `DemandeAdoptionCréée` ; MS7 envoie l'accusé de réception.
3. Le personnel consulte le score de compatibilité (`GET /api/demandes/{id}/score`) puis accepte la demande (`PUT /api/demandes/{id}/statut`). MS2 publie `DemandeAdoptionAcceptée` ; MS7 prévient le client.
4. Le personnel finalise l'adoption. MS2 appelle MS1 (`PATCH /api/animaux/17/statut` avec ADOPTE) ; MS1 contrôle la transition, écrit l'historique et publie `StatutAnimalModifié`.
5. MS6 reçoit `StatutAnimalModifié` : il annule les visites à venir de l'animal et prévient les visiteurs.
6. MS2 enregistre la demande à FINALISEE et publie `AdoptionFinalisée`.
7. MS3 reçoit l'événement et termine le placement éventuel ; MS5 termine les parrainages actifs et publie `ParrainageTerminé` ; MS7 envoie les félicitations au client et les e-mails de fin de parrainage.

À aucun moment un service ne lit la base d'un autre : MS1 reste la seule source de vérité sur l'animal, et les autres services réagissent par appel REST ou par événement.

8. Technologies choisies

La stack respecte celle imposée par le module MT-41 ; les choix complémentaires (RabbitMQ, OpenFeign, React) répondent aux chapitres sur la communication entre services et au front-end.

Besoin	Technologie	Justification
Microservices	Java 17, Spring Boot 3.5, Spring Data JPA, Bean Validation, Maven	Stack imposée ; même structure que l'exemple vu en cours
API Gateway	Spring Cloud Gateway	Stack imposée ; point d'entrée unique, routage et filtre de sécurité
Découverte de services	Eureka	Stack imposée ; les services se trouvent par leur nom
Sécurité	Keycloak (OIDC, JWT)	Stack imposée ; gestion des comptes, des rôles et des groupes
Bases de données	MySQL (MS1 à MS6), MongoDB (MS7), H2 pour les tests	Une base par service ; MongoDB pour le service commun (technologie avancée)
Communication synchrone	OpenFeign, Resilience4j	Appels REST déclaratifs entre services ; délai maximal et coupe-circuit pour répondre 503 proprement
Communication asynchrone	RabbitMQ (Spring AMQP)	Événements découplés, files durables, routage par clé (exchange topic)
Documentation d'API	Swagger / OpenAPI (springdoc)	Stack imposée ; chaque service expose ses endpoints
Conteneurisation	Docker, Docker Compose	Stack imposée ; un fichier démarre toute l'application
Frontend	React	Application web qui consomme les API par la Gateway
Code et suivi	Git, GitHub, GitHub Projects	Dépôt public, contributions visibles, suivi des tâches

9. Répartition des tâches

Chaque membre est responsable d'un microservice, qu'il conçoit, développe, documente (Swagger) et présente. Le microservice commun MS7 et l'infrastructure sont réalisés par toute l'équipe.

Service	Responsable	À réaliser
MS1 Animaux		Fiches, statuts, photos, historique, statistiques ; événements AnimalCréé et StatutAnimalModifié
MS2 Demandes d'adoption		Profils, demandes, score de compatibilité, finalisation ; appels vers MS1 ; 4 événements
MS3 Familles d'accueil		Familles, capacité, placements ; appels vers MS1 ; événements de placement ; écoute d'AdoptionFinalisée
MS4 Suivi vétérinaire		Dossiers, consultations, vaccinations, traitements ; tâche de rappel ; 2 événements
MS5 Dons et parrainages		Donateurs, dons, reçus, parrainages, statistiques ; 3 événements ; écoute d'AdoptionFinalisée
MS6 Visites et rendez-vous	Kmar	Créneaux, visites, événements, inscriptions, liste d'attente ; 4 événements ; écoute de StatutAnimalModifié
MS7 Notifications (commun)	Toute l'équipe	Squelette du service, puis un écouteur et un gabarit par membre pour les événements de son service

Tâches communes

Tâche commune	Organisation
API Gateway, routes et sécurité	Un responsable désigné dans GitHub Projects ; chaque membre déclare les routes de son service
Eureka et configuration des services	Un responsable désigné ; chaque membre enregistre son service
Keycloak : royaume, rôles, groupe « veterinaire »	Un responsable désigné ; revue par toute l'équipe
RabbitMQ et Docker Compose	Un responsable désigné ; chaque membre fournit le Dockerfile de son service
Frontend React	Découpé par service : chaque membre réalise les écrans de son microservice
Dossier technique et présentation	Parties réparties équitablement entre les membres ; chacun présente son propre microservice (1 minute) ; un membre présente MS7

10. Organisation du dépôt Git et gestion de projet

10.1 Dépôt

Le dépôt public `refugia-4DAS_I13-InSync` est hébergé sur GitHub ; l'enseignant et tous les membres de l'équipe y sont ajoutés comme collaborateurs.

```
refugia-4DAS_I13-InSync/
├── Backend/
│   ├── ms1-animaux/
│   ├── ms2-demandes-adoption/
│   ├── ms3-familles-accueil/
│   ├── ms4-suivi-veterinaire/
│   ├── ms5-dons-parrainages/
│   ├── ms6-visites-rendezvous/
│   └── ms7-notifications/
├── Frontend/
├── Documentation/
│   ├── Presentation/
│   └── Diagrams/
└── README.md
```

Le dossier technique (ce document) est placé dans `Documentation/`, la présentation dans `Documentation/Presentation/` et les diagrammes (architecture globale, cas d'utilisation, diagrammes de classes et de séquence de chaque service) dans `Documentation/Diagrams/`. Chaque dossier de service suit la structure décrite en 5.3.

10.2 Méthode de travail

- La branche `main` contient la version stable : on n'y pousse pas directement.
- Une branche par fonctionnalité, nommée `feature/<service>-<sujet>` (exemple : `feature/ms6-creneaux`).
- Fusion dans `main` par Pull Request relue par un autre membre.
- Chaque membre travaille dans le dossier de son service, ce qui limite les conflits ; les contributions de chacun restent visibles dans l'historique des commits.
- Messages de commit courts, au présent, préfixés par le service (exemple : `ms6: ajoute la création de créneaux`).

10.3 Outil de gestion de projet

Le suivi des tâches se fait dans un tableau **GitHub Projects** commun à l'équipe, lié au dépôt et public. Il comporte quatre colonnes : **À faire**, **En cours**, **En revue**, **Terminé**. Chaque tâche est une carte (ou une issue) rattachée à un service et assignée à un membre ; chaque membre déplace ses cartes au fil de son avancement. Le tableau sert aussi de preuve de la répartition du travail.

11. Planning prévisionnel

Le planning suit les dix séances du module : la conception est validée en séance 4 (jalon 1), puis la réalisation avance au rythme des chapitres du cours jusqu'à la soutenance en séance 10 (jalon 2).

Séance	Objectifs	Livrables
S1 à S3	Former l'équipe, choisir le sujet, découper en microservices, définir les contrats d'API	Idée du projet, découpage, premiers contrats
S4 · Jalon 1	Valider la conception	Dépôt Git, dossier technique, présentation, diagramme d'architecture globale
S5	API Gateway, Eureka, CRUD de chaque service, Swagger, début du frontend	Services qui démarrent, s'enregistrent dans Eureka et sont joignables par la Gateway ; MS1 en priorité

Séance	Objectifs	Livrables
S6	Communication synchrone : appels OpenFeign vers MS1, délai et coupe-circuit	Réservation de visite, don, placement, demande avec vérification de l'animal
S7	Communication asynchrone : RabbitMQ, publication et écoute des événements, MS7	17 événements, notifications de bout en bout
S8	Sécurité avec Keycloak : rôles, jetons JWT, droits par endpoint	Connexion, trois rôles, groupe « veterinaire »
S9	Dockerisation, Docker Compose, finalisation du frontend, tests	Démarrage de toute l'application par un seul fichier
S10 · Jalon 2	Démonstration et soutenance	Application complète, README de lancement, code de chaque service

Le planning répond aussi aux huit critères d'évaluation du module : choix architecturaux (sections 3 et 4), outil de gestion de projet (section 10), choix fonctionnels et données (section 6), résultats d'exécution (S9 et S10), technologies respectées (section 8), structuration du code (section 5.3), communication entre les microservices (section 7) et richesse des API (section 6).

12. Principaux risques

Risque	Conséquence	Réponse prévue
Contrats d'API ou noms qui divergent entre services	Appels et événements qui ne s'accordent pas à l'intégration	Conventions communes (section 5) et catalogue d'événements (section 7.2) validés ensemble ; Swagger pour vérifier
MS1 en retard	Tous les autres services en dépendent	MS1 en priorité à la séance 5 ; données de test partagées ; réponse 503 prévue côté appelant
Événements perdus ou traités deux fois	Statuts ou notifications incohérents	Files durables, enveloppe commune avec `eventId`, idempotence chez le consommateur
Sécurité Keycloak mise en place tard (S8)	Régressions sur les endpoints déjà écrits	Règles de rôle écrites dans les contrats dès maintenant ; configuration de sécurité isolée dans `common/config`
Intégration Docker tardive (S9)	Services qui ne démarrent pas ensemble	Dockerfile par service dès que le service démarre ; Docker Compose partiel (bases, RabbitMQ) dès la S5
Retard ou absence d'un membre	Un service manquant	Suivi dans GitHub Projects, point d'équipe hebdomadaire, relecture croisée des Pull Requests
Conflits Git	Perte de temps, code écrasé	Un dossier par service, une branche par fonctionnalité, Pull Request obligatoire
Service commun MS7 laissé à la fin	Notifications absentes à la démonstration	Squelette posé à la S5 ; chaque membre ajoute son écouteur en même temps que ses événements

13. Conclusion

Refugia répond à un besoin réel de centralisation d'un refuge par sept microservices autonomes : chacun a sa responsabilité, sa base de données et son contrat d'API, et tous partagent les mêmes conventions de rôles, d'erreurs et d'événements. L'architecture respecte la stack imposée (Spring Boot, Gateway, Eureka, Keycloak, Docker) et ajoute une communication asynchrone par RabbitMQ qui évite de bloquer un service quand un autre est arrêté.

La conception est cohérente de bout en bout : le scénario d'adoption traverse six services sans qu'aucun ne lise la base d'un autre, et MS1 reste la seule source de vérité sur l'animal. Les prochaines séances transforment ces contrats en code, en suivant le planning de la section 11.

Annexe A · Récapitulatif des services

Service	Endpoints	Événements publiés	Événements reçus	Base
MS1 Animaux	10	2	0	MySQL
MS2 Demandes d'adoption	12	4	0	MySQL
MS3 Familles d'accueil	11	2	1	MySQL
MS4 Suivi vétérinaire	14	2	0	MySQL
MS5 Dons et parrainages	15	3	1	MySQL
MS6 Visites et rendez-vous	15	4	1	MySQL
MS7 Notifications	5	0	17	MongoDB
Total	**82**	**17**		

Annexe B · Glossaire

Terme	Définition
API Gateway	Point d'entrée unique de l'application : il reçoit toutes les requêtes et les envoie au bon service
Eureka	Registre où chaque service s'enregistre, pour que les autres le trouvent par son nom
Keycloak / OIDC / JWT	Serveur d'authentification ; protocole de connexion ; jeton signé qui contient l'identité et les rôles de l'utilisateur
RabbitMQ / exchange topic	Broker de messages ; un exchange topic oriente chaque message vers les files qui s'intéressent à sa clé de routage
OpenFeign	Bibliothèque qui permet d'appeler un autre service REST comme une simple interface Java
Coupe-circuit (Resilience4j)	Mécanisme qui arrête d'appeler un service qui ne répond plus et renvoie vite une erreur 503
Idempotence	Propriété d'un traitement qui donne le même résultat s'il est exécuté deux fois sur le même message
DTO	Objet qui représente le contenu d'une requête ou d'une réponse de l'API, distinct de l'entité enregistrée en base
Microservice commun	Service développé par toute l'équipe (ici MS7 Notifications)